Final Year Design Project

IoT-Based Smart Home Automation System Using ESP32

By

Mohsin Saleem BCS22006

Abdul Rehman Tahir BCS22016

Rehan Ali BCS22021

Muhammad Owais BCS22022

Wasif Ali BCS22024

Under the supervision of

Dr. Salman Naseer

Bachelor of Science in Computer Science (2022-2026)

DEPARTMENT OF INFORMATION TECHNOLOGY,

UNIVERSITY OF THE PUNJAB,

GUJRANWALA CAMPUS, GUJRANWALA

IoT-Based Smart Home Automation System Using ESP32

A project presented to

University of the Punjab, Lahore

In partial fulfilment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Mohsin Saleem	**BCS22006**
Abdul Rehman Tahir	**BCS22016**
Rehan Ali	**BCS22021**
Muhammad Owais	**BCS22022**
Wasif Ali	**BCS22024**

DEPARTMENT OF INFORMATION TECHNOLOGY,

UNIVERSITY OF THE PUNJAB,

GUJRANWALA CAMPUS, GUJRANWALA

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Signature: ----------

Mohsin Saleem (BCS22006)

Signature: ----------

Muhammad Owais (BCS22021)

Signature: ----------

Abdul Rehman Tahir (BCS22016)

Signature: ----------

Wasif Ali (BCS22024)

Signature: ----------

Rehan Ali (BCS22021)

CERTIFICATE OF APPROVAL

It is to certify that the final year design project (FYDP) of BSCS "IoT-Based Smart Home Automation System Using ESP32" was developed by **MOHSIN SALEEM (BCS22006), ABDUL REHMAN TAHIR (BCS22016), REHAN ALI (BCS22021), MUHAMMAD OWAIS (BCS22022)** and **WASIF ALI (BCS22024)** under the supervision of "**DR. SALMAN NASEER**" in my opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science.

Signature: ----------

FYDP Supervisor

Signatures (Faculty Advisory Committee (FAC)

Signatures			
Name			
	FAC1	FAC2	FAC3

Signature: ----------

Head of FYDP Coordination Office

Signature: ---------- Dated: _____

Chairperson, Department of Information Technology

Executive Summary

The project presents the design and implementation of a **Smart Home Automation System** that leverages modern IoT technologies to provide real-time monitoring and control of household appliances. The primary objective was to develop a system that enables users to remotely manage devices such as lights, fans, and water pumps, while simultaneously monitoring environmental parameters like temperature, humidity, and water level. The solution addresses the growing need for energy efficiency, convenience, and secure remote access in residential environments.

The methodology integrates three core components:

1. **ESP32 microcontroller programmed via Arduino IDE** to interface with sensors and actuators.
2. **Firebase Realtime Database** serving as the backend service, ensuring seamless cloud communication and synchronization between devices.
3. **Flutter mobile application** acting as the user interface layer, providing an intuitive dashboard for device control and sensor visualization.

The ESP32 continuously exchanges data with Firebase, while the Flutter app interacts with the same backend to issue commands and retrieve sensor readings. This architecture eliminates the need for a custom backend, relying instead on Firebase's robust cloud infrastructure for real-time updates and secure data handling.

The project concludes that the integration of ESP32, Firebase, and Flutter offers a scalable and efficient solution for smart home automation. The system successfully demonstrates reliable real-time communication, cross-platform accessibility, and user-friendly control. Its implications extend to broader IoT applications, where cloud-based services and custom user interfaces can be combined to deliver flexible, cost-effective automation solutions.

Keywords: Smart Home Automation, ESP32, Firebase, Flutter, IoT

Acknowledgement

First and foremost, we are profoundly grateful to **Allah Almighty**, whose countless blessings, guidance, and mercy enabled us to complete this Final Year Project successfully. Without His will and support, none of our efforts would have been fruitful.

We would like to express our deepest gratitude to our **Project Supervisor**, whose guidance, encouragement, and constructive feedback were invaluable throughout the course of this Final Year Project. His expertise and continuous support helped us refine our ideas, overcome technical challenges, and achieve the objectives of our work.

We are also sincerely thankful to the **faculty members** and **the Department of Information Technology** for providing the academic foundation, resources, and motivation that enabled us to carry out this project successfully. Their commitment to fostering innovation and research created the environment in which this project could thrive.

Our appreciation extends to our **friends and peers**, who offered valuable suggestions, shared knowledge, and provided moral support during the development and testing phases. Their collaboration and encouragement made the journey smoother and more rewarding.

Finally, we owe special thanks to our **families**, whose patience, understanding, and constant encouragement gave us the strength to complete this project. Their support has been a source of inspiration and determination throughout our academic journey.

Signature: ----------

Mohsin Saleem (BCS22006)

Signature: ----------

Muhammad Owais (BCS22021)

Signature: ----------

Abdul Rehman Tahir (BCS22016)

Signature: ----------

Wasif Ali (BCS22024)

Signature: ----------

Rehan Ali (BCS22021)

Abbreviations

API	Application Programming Interface – Functions and protocols enabling communication between software components.
APK	Android Package Kit – File format used to distribute and install Android applications.
BL	Backlog List – List of requirements under consideration.
DHT	Digital Humidity and Temperature Sensor – Sensor used for temperature-based automation.
ESP32	Espressif Systems 32-bit Microcontroller – Low-cost microcontroller with Wi-Fi and Bluetooth used in IoT projects.
FT	Functional Test – Test case verifying functional requirements.
FYP	Final Year Project – Capstone academic project undertaken in the final year of study.
HTTP	Hypertext Transfer Protocol – Protocol used for communication between web clients and servers.
IDE	Integrated Development Environment – Software application providing tools for programming, debugging, and compiling code.
IoT	Internet of Things – Network of physical devices connected to the internet for data exchange and automation.
IT	Integration Test – Test case verifying combined modules.
JSON	JavaScript Object Notation – Lightweight data format used for Firebase backup and restore.
LDR	Light Dependent Resistor – Sensor used for curtain automation based on ambient light.
LM393	Comparator Module – Used for fire/smoke detection with sensors.
MQ135	Gas Sensor – Detects smoke and harmful gases in smart home safety systems.
MQTT	Message Queuing Telemetry Transport – Lightweight messaging protocol often used in IoT systems.
PIR	Passive Infrared Sensor – Detects motion for light automation.
PT	Performance Test – Test case verifying system performance under load.
SRS	Software Requirements Specification – Document outlining functional and non-functional requirements.
UI	User Interface – Visual and interactive part of the application.
UT	Unit Test – Test case verifying individual modules.

Table of Contents

List of Figures

List of Tables

Chapter 1
Introduction

1. Introduction

This chapter provides an overview of the Smart Home Automation project. It introduces the motivation behind the system, outlines the challenges faced in traditional household management, and explains how the proposed solution contributes to achieving the overall project goals. By presenting the problem statement and its corresponding solution, this chapter establishes the foundation for the project and demonstrates how the integration of embedded hardware, cloud services, and a mobile interface can deliver energy efficiency, convenience, and modern IoT-based living.

1.1 Problem Statement

Households today face increasing challenges in managing appliances efficiently due to rising energy costs, demand for convenience, and the need for secure remote access. Traditional systems rely on manual operation, which often results in wasted electricity, higher bills, and limited control over devices such as lights, fans, and water pumps. Users cannot monitor real-time environmental parameters like temperature, humidity, or water levels, making it difficult to automate appliances based on actual conditions.

Existing smart home solutions are either expensive, proprietary, or lack flexibility for customization. Many require closed ecosystems or complex backend development, which makes them inaccessible to students, researchers, and budget-conscious users. Furthermore, these systems often fail to provide seamless integration between hardware sensors, cloud services, and mobile applications, leaving a gap between IoT capabilities and everyday household needs.

The absence of a low-cost, scalable, and cloud-connected solution creates inefficiencies in energy management and limits user convenience. Without automation modes, energy tracking, or real-time alerts, households cannot optimize their usage or respond to peak electricity hours effectively.

This project is being developed to address these challenges by creating a smart home automation system that integrates ESP32 microcontrollers, Firebase Realtime Database, and a Flutter-based mobile application. The system aims to provide real-time monitoring, remote control, and automation features, ensuring energy efficiency, convenience, and accessibility. By bridging the gap between embedded hardware, cloud services, and user-friendly interfaces, the project solves the problem of inaccessible and inefficient home automation, offering a practical and affordable IoT solution for modern households.

1.2. Problem Solution

To overcome the identified challenges, the project proposes a smart home automation system that combines embedded hardware, cloud services, and a custom mobile interface. The ESP32 microcontroller acts as the hardware backbone, interfacing with sensors and relays to control appliances and collect environmental data. The firmware

is developed in Arduino IDE, ensuring compatibility with widely available libraries and modules.

Firebase Realtime Database serves as the backend service, providing secure cloud communication and synchronization. This eliminates the need for a separate backend server, reducing complexity while ensuring real-time data exchange between devices and the mobile application.

The Flutter mobile application functions as the frontend, offering an intuitive user interface for device control and sensor visualization. It communicates directly with Firebase, allowing users to issue commands, view sensor readings, and monitor energy usage seamlessly.

Objectives of the Solution

- Enable remote control of appliances such as lights, fans, and water pumps.
- Implement automation modes (manual, auto, schedule) for flexible device control.
- Track energy consumption and generate warnings when usage exceeds limits.
- Alert users during peak electricity hours using buzzer notifications.
- Reset usage data monthly to align with billing cycles.
- Deliver a cross-platform mobile application with a clean and responsive UI.
- Ensure scalability so additional devices and sensors can be integrated easily.
- Maintain affordability by using open-source tools and low-cost hardware.

By integrating these components, the project achieves a cost-effective, reliable, and scalable smart home automation solution. It demonstrates how IoT technologies can be applied to everyday life, improving energy efficiency, convenience, and accessibility for users.

1.3. Objectives of the Proposed System

The proposed system is designed to achieve practical and measurable objectives that directly contribute to energy efficiency, affordability, and user convenience. Each objective is realistic within the constraints of time, budget, and resources, and aligns with the overall project goals. The objectives are documented clearly, prioritized for impact, and focused on improving the user experience.

Objectives

- **Enable remote appliance control** – Allow users to switch ON/OFF the four relay-connected devices (fan, light, curtain motor, water pump) from anywhere using the mobile application.
- **Support automation modes** – Provide **manual, auto, and schedule modes** for flexible device control.

- **Schedule mode** – Allow users to define ON/OFF times for each of the four appliances, enabling routine automation.
- **Fire and smoke detection** – Continuously monitor LM393 fire sensor and MQ135 smoke sensor; trigger buzzer alerts when hazards are detected.
- **Peak hour alerts** – Activate buzzer notifications when high-consumption devices run during peak electricity hours.
- **Cross-platform mobile application** – Deliver a Flutter-based app with a responsive and intuitive UI for Android and iOS.
- **Secure cloud communication** – Utilize Firebase Realtime Database for reliable, authenticated, and synchronized data exchange.
- **User login and authentication** – Provide secure account creation and login using Firebase Authentication.
- **Scalability** – Ensure the system can integrate additional sensors or devices without major redesign.
- **Affordability** – Use low-cost hardware (ESP32, relays, sensors) and open-source tools to keep the solution budget-friendly.

1.4. Scope

The Smart Home Automation project is carefully bounded to ensure that its deliverables remain achievable within the academic timeline, available resources, and chosen technologies. The scope emphasizes electricity usage tracking, monthly unit limits, and appliance control through a mobile interface, while deliberately excluding features that would introduce unnecessary complexity or exceed project constraints. This logical flow ensures alignment with objectives, prevents scope creep, and keeps the system focused on user-centric functionality.

Inclusion

- Remote ON/OFF control of appliances such as lights, fans, and water pumps.
- Usage screen displaying total electricity units consumed.
- Monthly consumption limit set to **200 units by default**, with user customization available.
- Automatic reset of allowed units back to 200 and consumed units back to 0 at the start of each new month.
- Device-wise usage tracking and warnings when consumption exceeds the set limit.
- Automation modes including manual, auto, and schedule.
- Buzzer alerts during peak electricity hours.
- Cross-platform Flutter mobile application connected to Firebase backend.

- ESP32 firmware implementing device control, usage tracking, and monthly reset logic.
- Documentation including system architecture, class diagrams, and user manual.

Exclusion

- Real-time sensor monitoring of temperature, humidity, or water levels.
- Integration with third-party smart home ecosystems (e.g., Alexa, Google Home).
- Advanced AI-based predictive analytics beyond usage tracking.
- Complex hardware expansion beyond ESP32 and basic sensors/relays within the project timeline.

1.5. System Components

The proposed system is divided into three main modules: **Client Mobile App**, **Cloud Backend**, and **Hardware Controller (ESP32 Firmware)**. Each module is responsible for specific functionality and together they form the complete smart home automation solution.

1.5.1. Module 1: Client Mobile App

- User login and authentication.
- Appliance ON/OFF control via interactive buttons.
- Usage screen displaying total consumed units.
- Option to set or change monthly unit limit (default 200 units).
- Automatic reset of allowed units and consumed units at the start of each month.
- Device-wise usage tracking.
- Notifications when consumption approaches or exceeds the set limit.
- Alerts during peak electricity hours.
- Clean, responsive UI built with Flutter for Android and iOS.

1.5.2. Module 2: Cloud Backend (Firebase Realtime Database)

- Secure storage of appliance states and usage data.
- Synchronization between ESP32 hardware and mobile app.
- Authentication and user data management.
- Real-time updates for usage screen and alerts.
- Scalability to support multiple devices and users.

1.5.3. Module 3: Hardware Controller (ESP32 Firmware)

- Control of appliances through relay modules.

- Measurement of electricity usage per device.
- Communication with Firebase for data logging and synchronization.
- Implementation of monthly reset logic (allowed units = 200, consumed units = 0).
- Triggering buzzer alerts during peak hours.
- Support for automation modes (manual, auto, schedule).
- Usage monitoring and warning generation when limits are exceeded.
- Billing reset at the start of each new month.

1.6. Related System Analysis / Literature Review

Several smart home automation systems exist in the market, but most suffer from limitations such as high cost, lack of customization, or incomplete usage tracking. The proposed project contributes by offering a low-cost, flexible, and usage-focused solution that aligns with household needs and academic feasibility as shown in ***Table 1***.

Table 1: Related System Analysis with Proposed Project Solution

Application Name	Weakness	Proposed Project Solution
Google Nest	Expensive ecosystem, requires proprietary hardware, limited customization for usage tracking.	Provides low-cost ESP32 hardware with customizable monthly unit limits and usage screen.
Samsung SmartThings	Complex setup, dependency on third-party devices, lacks simple monthly consumption tracking.	Offers a straightforward Flutter app with built-in usage monitoring and monthly reset logic.
Amazon Alexa Smart Home	Focuses on voice control, but lacks detailed energy usage tracking and billing reset features.	Integrates usage tracking, warnings, and billing reset (200 units default) alongside appliance control.
Xiaomi Mi Home	Affordable but limited in automation modes and lacks peak hour alerts.	Implements manual, auto, and schedule modes plus buzzer alerts during peak electricity hours.

1.7. Vision Statement

For household users and families **who** seek an affordable, convenient, and efficient way to manage appliances and electricity consumption, **the** Smart Home Automation System **is** an IoT-based energy management and appliance control solution **that** provides remote appliance control, customizable monthly usage limits, automatic resets, **and** peak hour alerts to improve energy efficiency and user satisfaction. **Unlike**

proprietary ecosystems such as Google Nest or Samsung SmartThings that are costly and complex**, our product** offers a low-cost, scalable, and user-friendly solution built on ESP32, Firebase, and Flutter, delivering practical automation without unnecessary complexity or high expenses.

1.8. System Limitations and Constraints

Following are the system limitations and constraints of our project.

1.8.1. Limitations

- **Hardware dependency:** The system relies on ESP32 microcontrollers and basic sensors; advanced hardware integration (e.g., AI-enabled devices) is beyond current scope.
- **Internet connectivity:** Continuous cloud synchronization requires stable Wi-Fi; performance may degrade in areas with poor connectivity.
- **Third-party ecosystem integration:** Compatibility with platforms like Alexa or Google Home is not included, limiting interoperability.
- **Scalability limits:** While expandable, large-scale deployments may require more robust cloud infrastructure than Firebase.
- **Environmental factors:** Sensor accuracy may vary due to temperature fluctuations, electrical noise, or hardware wear.

1.8.2. Constraints

- **Time:** Project completion is restricted to the academic semester.
- **Technology:** Limited to ESP32, Firebase Realtime Database, and Flutter framework.
- **Budget:** Use of low-cost hardware and open-source tools to maintain affordability.
- **Scope:** Focused only on appliance control, usage tracking, monthly reset, and alerts; excludes advanced predictive analytics or AI-driven automation.
- **Resources:** Development and testing constrained by available lab equipment and team size.
- **Priorities:** Critical features (usage tracking, monthly reset, appliance control) are prioritized over optional scalability features.

1.9. Tools and Technologies

The Smart Home Automation project uses ESP32 DevKit V1 as the core hardware, supported by sensors, relays, and a buzzer for appliance control and monitoring. Firmware is developed in C/C++ using the Arduino IDE, while Flutter 3.41.9 powers the cross-platform mobile app. Firebase Realtime Database ensures secure cloud synchronization, with APIs and SDKs enabling communication between hardware and app. GitHub is used for version control, making the overall stack affordable, reliable, and scalable. It is shown in ***Table 2***.

Table 2: Tools and Technologies for Proposed Project

	Tools	**Version**	**Rationale**
Tools And Technologies	Microcontroller (ESP32 DevKit V1)	Latest stable	Provides Wi-Fi and GPIO support for appliance control and sensor integration.
	Sensors (DHT11, PIR, LDR, Water Level Sensor)	Standard modules	Enables temperature, motion, light, and water level monitoring for automation.
	Relay Module (4-Channel Relay)	Standard	Controls appliances (lights, fans, pumps) via ESP32.
	Buzzer (Piezoelectric Buzzer)	Standard	Provides peak hour alerts.
	Technology	**Version**	**Rationale**
	Programming Language (C/C++ (Arduino Core for ESP32)	Latest stable	Used for embedded programming of ESP32 firmware.
	Mobile App Framework (Flutter 3.41.9)	Current stable (3.41.9)	Cross-platform mobile app development for Android/iOS.

	Backend Database (Firebase Realtime Database)	Current stable	Provides secure cloud storage and real-time synchronization.
	APIs/SDKs (Firebase SDK for Flutter, Arduino Firebase Library)	Latest	Enables communication between ESP32, Firebase, and Flutter app.
	IDE/Tools Arduino IDE 1.8.19, Visual Studio Code	Latest	Used for firmware development and app coding.
	Version Control (GitHub)	Current	Manages source code, documentation, and collaboration.

1.10. Project Deliverables

The following deliverables will be produced during the course of the project and appended to the FYP report:

- **Project Proposal –** Initial project plan with objectives, scope, milestones, and constraints.
- **Software Requirements Specification (SRS) –** Detailed requirements of the system including functional and non-functional aspects.
- **System Design Specification (SDS) –** Architecture diagrams, class diagrams, and module design details.
- **Prototypes –** Early versions of mobile app and firmware demonstrating core features.
- **Final Project System** – Complete Smart Home Automation solution (ESP32 firmware, Firebase backend, Flutter mobile app).
- **Project Report –** Comprehensive documentation of analysis, design, implementation, testing, and evaluation.
- **Appendices –** Supporting materials such as code snippets, test cases, user manual, and additional diagrams.

1.11. Project Planning

The project plan is represented through a **Gantt Chart**, which outlines the timeline of activities, milestones, and resource allocation. It shows how tasks such as requirements gathering, design, implementation, testing, and documentation are scheduled across the project duration. Milestones like completion of design models, integration testing, and final deployment are highlighted to track progress. Resources including the ESP32 controller, sensors, relay modules, Firebase services, and mobile app development tools are allocated to relevant tasks.

Figure *1* provides a clear visualization of dependencies, workload distribution, and deadlines, ensuring systematic progress and effective monitoring.

Figure 1: Gantt Chart

1.12. Summary

This chapter sets the stage for the Smart Home Automation project. It explains the problems with traditional household management — high energy costs, lack of automation, and limited monitoring — and introduces a low-cost IoT solution using ESP32, Firebase, and a Flutter mobile app. The objectives focus on remote appliance control, automation modes, usage tracking, alerts, and monthly resets.

The scope defines what's included (appliance control, usage monitoring, alerts) and what's excluded (third-party ecosystems, advanced AI). System components are divided into the mobile app, Firebase backend, and ESP32 firmware. Tools and technologies like Arduino IDE, Flutter, and GitHub are documented, while deliverables

include design specs, prototypes, and the final system. A Gantt Chart outlines project planning and milestones.

In short, Chapter 1 provides the foundation by clarifying the problem, solution, objectives, scope, and plan — showing how the project will deliver an affordable, scalable, and practical smart home automation system.

Chapter 2
Requirements Analysis

2. Analysis

This chapter presents the analysis of the proposed Smart Home Automation system. It identifies the different classes of users who will interact with the system, outlines their characteristics, and explains the techniques used to derive system requirements. The analysis ensures that the system design is user-focused, practical, and aligned with project objectives.

2.1. User Classes and Characteristics

The system will be used by different categories of users, each with distinct needs and technical familiarity. The table below summarizes the anticipated user classes and their pertinent characteristics:

User Class	**Characteristics**
Home Residents	Primary users; operate appliances via mobile app; expect simplicity, reliability, and real-time control.
System Administrator	Responsible for setup, maintenance, and troubleshooting; requires technical knowledge of IoT devices and Firebase backend.
Guests/Visitors	Occasional users; limited access to certain features; need intuitive interface without complex setup.
Developers/Researchers	Secondary users; may extend or customize system; require access to documentation, source code, and APIs.

2.2. Requirement Identifying Technique

For this project, the **Use Case technique** has been selected as the requirement-identifying method. Since the Smart Home Automation system is interactive and involves direct user control through a mobile application, use cases provide a structured and effective way to capture functional requirements.

Use cases clearly define the actors (such as Home Resident, Administrator, Guest User) and their interactions with the system. Each use case is represented in a diagram and described in a fully dressed format, including preconditions, main flow, alternative flows, and postconditions. This ensures that requirements are documented in a user-centric manner and can be easily validated against project objectives.

The use case approach also allows direct mapping of each scenario to functional requirements, ensuring traceability and completeness. By applying this technique, the analysis phase produces both a **use case diagram** and **detailed use case descriptions**, which together form the foundation for the functional requirements specification in the next chapter.

2.3. Functional Requirements

This section defines the functional behaviour of the IoT-Based Smart Home Automation System. After registering and logging in through Firebase Authentication, users can control all Wi-Fi-enabled appliances connected to the ESP32, monitor real-time device states, and enable Automatic Mode for sensor-based automation. The system follows modern cloud-connected smart-home models, enabling remote control, real-time synchronization, and intelligent automation. Features align with cloud-connected smart-home models enabling remote appliance control and energy automation (Ahmad, 2023; Sharma, 2022; Nguyen, Patel, & Kumar, 2025).

2.3.1 User Registration (Functional Requirement: FR-1)

This feature allows a new user to create an account using Firebase Authentication. As shown in **Table *3,*** the system verifies the entered information and creates a unique user profile if all conditions are met.

Table 3: Functional Requirement - User Registration

Identifier	FR-1
Title	User Registration
Requirement	The *User* shall be able to create an account by providing a valid email address and password through Firebase Authentication. The system shall verify the entered information and create a unique user profile if all conditions are met.
Source	**Project Objective:** Provide secure user login and authentication through Firebase.
Rationale	Ensures that only authenticated users can access and control smart devices.
Business Rule	Password must be at least 8 characters long and contain at least one letter and one number.
Dependencies	--
Priority	High

2.3.2 User Login (Functional Requirement: FR-2)

This feature enables registered users to access their account using correct login credentials. As shown in **Table 4**, the system verifies details before granting access.

Table 4: Functional Requirement - User Login

Identifier	FR-2
Title	User Login
Requirement	The system shall allow a registered user to log in using Firebase Authentication by entering a valid email and password.
Source	Objective — Ensure authenticated access to appliance control and automation features.
Rationale	Ensures secure access to device control and automation features.
Business Rule	After 3 failed attempts, the account shall be locked for 5 minutes.
Dependencies	FR-1 (User Registration)
Priority	High

2.3.3 Device Onboarding (Functional Requirement: FR-3)

This requirement enables users to add ESP32-controlled appliances to their account. As shown in

Table 5, devices must be connected to the same Wi-Fi network during onboarding.

Table 5: Functional Requirement - Device Onboarding (Pairing)

Identifier	FR-3
Title	Relay Device Onboarding

Requirement	The *System* shall automatically display the four appliances connected to the ESP32 via the 4-channel relay. The user shall be able to view and control only those devices.
Source	**Objective —** Enable remote control of four relay-connected appliances via ESP32 and Firebase.
Rationale	Since devices are physically wired to the relay, onboarding is automatic and requires no manual user setup.
Business Rule	Only the four relay-connected appliances are visible in the app; users cannot add or remove devices.
Dependencies	FR-2 (User Login)
Priority	High

2.3.4 Remote Appliance ON/OFF Control (Functional Requirement: FR-4)

This requirement allows users to manually control appliances through the mobile app. Commands are sent to the ESP32 via Firebase. As shown in **Table 6**, the command is sent.

Table 6: Functional Requirement - Remote Appliance ON/OFF Control

Identifier	FR-4
Title	Remote Appliance Control (ON/OFF)
Requirement	The *User* shall be able to turn ON or OFF each of the four relay-connected appliances (Fan, Light, Curtain Motor, Water Pump) using the mobile application. The system shall send commands to Firebase, which updates the ESP32 device state.
Source	**Objective —** Allow remote ON/OFF control of appliances (fan, light, curtain motor, water pump).
Rationale	Core functionality for smart home management.
Business Rule	If a device is offline or unreachable, the app shall display an appropriate status message.
Dependencies	FR-3 (Relay Device Onboarding)
Priority	High

2.3.5 Automatic Mode Activation (Functional Requirement: FR-5)

This requirement enables users to activate Automatic Mode, allowing the ESP32 to make decisions based on sensor inputs as shown in **Table 7**.

Table 7: Functional Requirement – Automatic Mode Activation

Identifier	FR-5
Title	Automatic Mode Activation
Requirement	The *User* shall be able to enable or disable Automatic Mode from the mobile application. The system shall allow ESP32 to make decisions for the four relay-connected appliances based on sensor inputs.
Source	**Project Objective:** Support automation modes (manual, auto, schedule) for flexible device control.
Rationale	Allows the ESP32 to automatically control appliances using sensor inputs.
Business Rule	Automatic Mode overrides manual ON/OFF commands for supported appliances.
Dependencies	FR-3
Priority	High

2.3.6 Temperature-Based Fan Control (Functional Requirement: FR-6)

This requirement defines how the system uses temperature readings to automate fan control when Automatic Mode is active as shown in **Table 8**.

Table 8: Functional Requirement – Temperature Based Fan Control

Identifier	FR-6
Title	Temperature-Based Fan Control
Requirement	When Automatic Mode is active, the *System* shall automatically turn the fan ON or OFF based on temperature thresholds.
Source	**Project Objective -** Improve energy efficiency through sensor-based automation.

Rationale	Improves comfort and energy efficiency.
Business Rule	Threshold values may be predefined or user-adjustable.
Dependencies	FR-5
Priority	High

2.3.7 Motion-Based Light Automation (Functional Requirement: FR-7)

This requirement describes how the system uses motion detection to automate lighting as shown in **Table 9**.

Table 9: Functional Requirement – Motion-Based Light Automation

Identifier	FR-7
Title	Motion-Based Light Automation
Requirement	When Automatic Mode is active, the *System* shall automatically turn lights ON or OFF based on motion detection.
Source	**Project Objective:** Enhance convenience and reduce unnecessary energy usage
Rationale	Enhances convenience and reduces unnecessary energy usage.
Business Rule	Uses PIR sensor for occupancy detection.
Dependencies	FR-5
Priority	Medium

2.3.8 Light-Level Curtain Automation (Functional Requirement: FR-8)

As shown in **Table 10**, this requirement explains how the system uses ambient light levels to automate curtain movement.

Table 10: Functional Requirement – Light-Level Curtain Automation

Identifier	FR-8
Title	Light-Level Curtain Automation
Requirement	When Automatic Mode is active, the *System* shall automatically open or close curtains based on ambient light levels.
Source	**Project Objective:** Improve comfort and natural lighting efficiency using sensor inputs.
Rationale	Improves comfort and natural lighting efficiency.
Business Rule	Uses LDR sensor to detect day/night conditions.
Dependencies	FR-5
Priority	Medium

2.3.9 Water-Level Pump Automation (Functional Requirement: FR-9)

This requirement defines how the system manages water pump operation based on tank water levels as shown in **Table 11**.

Table 11: Functional Requirement – Water-Level Pump Automation

Identifier	FR-9
Title	Water-Level Pump Automation
Requirement	When Automatic Mode is active, the *System* shall automatically turn the water pump ON or OFF based on tank water level readings.
Source	**Project Objective:** Ensure efficient water usage and prevent overflow through automation.
Rationale	Prevents tank overflow and ensures efficient water usage.

Business Rule	Uses water-level sensor to detect minimum and maximum thresholds.
Dependencies	FR-5
Priority	High

2.3.10 Peak-Time Buzzer Alert (Functional Requirement: FR-10)

This requirement specifies how the system alerts users during predefined peak-time intervals as shown in **Table 12**.

Table 12: Functional Requirement – Peak-Timer Buzzer Alert

Identifier	FR-10
Title	Peak-Time Buzzer Alert
Requirement	The system shall activate a buzzer during predefined peak-time intervals.
Source	**Project Objective:** Alert users during peak electricity hours using buzzer notifications.
Rationale	Helps users manage energy usage during peak hours.
Business Rule	Peak-time ranges may be predefined or user-adjustable.
Dependencies	FR-3
Priority	Low

2.3.11 Communication Handling (Functional Requirement: FR-11)

This requirement defines how the system manages communication between the mobile app, Firebase, and ESP32 as shown in **Table 13** .

Table 13: Communication Handling

Identifier	FR-11
Title	Communication Handling

Requirement	The *System* shall use Firebase Authentication for login and shall communicate with ESP32 devices using Wi-Fi and Firebase Realtime Database.
Source	**Project Objective:** Ensure secure cloud communication via Firebase Realtime Database.
Rationale	Ensures secure login and reliable device communication.
Business Rule	Device control shall occur through Firebase-synchronized commands.
Dependencies	FR-1, FR-2
Priority	High

2.3.12 Schedule Mode (Functional Requirement: FR-12)

This requirement defines how the schedule mode will work. It is shown in **Table *14***.

Table 14: Functional Requirement - Schedule Mode

Identifier	FR-12
Title	Schedule Mode for Appliances
Requirement	The *User* shall be able to schedule ON/OFF times for each of the four relay-connected appliances (Fan, Light, Curtain Motor, Water Pump) through the mobile application. The system shall execute scheduled commands automatically via ESP32.
Source	**Project Objective:** Provide schedule mode for time-based appliance control.
Rationale	Enables users to automate appliances according to daily routines.
Business Rule	Scheduled commands override manual control only at the defined time slots.
Dependencies	FR-3, FR-4
Priority	High

2.3.13 Fire and Smoke Detection (Functional Requirement: FR-12)

To enhance safety within the smart home environment, the system integrates LM393 Fire Sensor and MQ135 Smoke Sensor. These sensors continuously monitor environmental conditions and provide early detection of fire or smoke hazards. When either sensor detects abnormal readings, the system immediately triggers the buzzer to alert occupants. This functionality ensures that the smart home is not only convenient and automated but also capable of responding to emergency situations, thereby protecting users and property from potential risks as shown in **Table *15***.

Table 15: Functional Requirement – Fire and Smoke Detection

Identifier	FR-13
Title	Fire and Smoke Detection with Buzzer Alert
Requirement	The *System* shall continuously monitor the LM393 Fire Sensor and MQ135 Smoke Sensor. If fire or smoke is detected, the buzzer shall automatically activate to alert the user.
Source	**Project Objective —** Enhance safety with LM393 fire sensor and MQ135 smoke sensor, triggering buzzer alerts during hazards.
Rationale	Provides immediate alerts to prevent hazards and ensure user safety.
Business Rule	Buzzer remains active until the hazard condition clears or is acknowledged in the app.
Dependencies	FR-3
Priority	High

2.4. Non-Functional Requirements

This section specifies non-functional requirements that define the quality attributes of the Smart Home Automation system. These requirements ensure reliability, usability, performance, security, and other quality factors beyond the functional capabilities described in Section 2.3.

2.4.1. Reliability (NFR-1)

1. The system shall achieve a **Mean Time Between Failures (MTBF) of at least 500 hours** under normal operating conditions.

2. In case of failure, the system shall attempt automatic recovery within 10 seconds.
3. The buzzer alert subsystem (fire/smoke detection) shall have redundant checks to minimize false negatives.

2.4.2. Usability (NFR-2)

4. The mobile application shall allow a new user to learn and operate basic functions within 10 minutes without external guidance.
5. Common tasks (e.g., turning a device ON/OFF, enabling schedule mode) shall require **no more than 3 interactions**.
6. The interface shall be accessible on both Android and iOS, with responsive design for different screen sizes.
7. Error messages shall be clear and actionable, guiding users to resolve issues.

2.4.3. Performance (NFR-3)

8. The system shall process and execute appliance control commands within **3 seconds** of user input under normal network conditions.
9. Scheduled tasks shall execute with a maximum deviation of **±2 second** from the defined time.
10. The mobile app shall load the dashboard within **3 seconds** under normal network conditions.

2.4.4. Security (NFR-4)

11. All communication between the mobile app, Firebase, and ESP32 shall be done using **HTTPS/TLS**.
12. User authentication shall be handled via Firebase Authentication, ensuring only authorized users can access devices.

2.4.5. Scalability (NFR-5)

13. The system shall support integration of additional sensors (e.g., temperature, humidity) without major redesign.
14. Database structure shall be flexible enough to handle multiple devices concurrently.

2.4.6. Portability (NFR-6)

15. The mobile application shall run on both Android and iOS platforms using Flutter.
16. The system shall support migration to other cloud platforms (e.g., AWS IoT, Google Firebase alternatives) with minimal changes.

2.4.7. Affordability (NFR-7)

17. The system shall use low-cost hardware (ESP32, relays, LM393, MQ135, PIR, LDR, water level sensors).

18. Open-source tools and frameworks shall be prioritized to minimize licensing costs.
19. The total hardware cost per unit shall remain within a predefined budget (e.g., under PKR 10,000).

2.5. External Interface Requirements

This section provides information to ensure that the system communicates properly with users and with external hardware or software elements. The Smart Home Automation system integrates multiple subcomponents - mobile application, cloud backend, and IoT hardware - requiring clear interface specifications.

2.5.1. User Interface Requirements

Following are the User Interface requirements for the project. **Table 16** shows the Ui requirements as follows:

Table 16: User Interface Requirements

Category	Property / Element	Specification / Value
Color Scheme	Primary Blue	#2196F3 – App bar, main buttons, active states
	Primary Green	#4CAF50 – Success states, ON indicators
	Amber	#FFC107 – Lights icon, warning highlights
	Purple	#9C27B0 – Curtains icon, premium features
	Background	#F5F9FF – Main screen background
	Card Background	#FFFFFF – Cards, dialogs, forms
	Text Primary	#333333 – Headings, main text
	Text Secondary	#757575 – Labels, subtitles, hints
	Error Color	#F44336 – Alerts, OFF states
	Success Color	#4CAF50 – ON states, success messages
Typography	Font Family	Inter (all text elements)
	App Title	22px Bold, Dark Gray
	Screen Heading	20px Bold, Dark Gray
	Section Title	18px Semi-Bold, Dark Gray
	Device Name	14px Semi-Bold, Dark Gray
	Body Text	14px Regular, Dark Gray
	Label Text	12px Regular, Medium Gray

	Button Text	16px Bold, White
Buttons	Primary Button	Gradient Blue to Green, White text, 16px Bold, 16px radius, shadow
	Secondary Button	Transparent/White, Blue border, Blue text, 14-16px font, 14-16px radius
	Text Button	Blue text, Semi-Bold, 13-14px font
Device Cards	Size	160×140px, White background, 14px radius, light shadow
	Icon Container	36×36px circle, device-specific color
	Device Color Mapping	Lights (Amber), Fan (Blue), Curtains (Purple), Water Motor (Green)
	Status Badges	ON: Green background/text; OFF: Red background/text
Switches	ON State	Device-specific active color
	OFF State	Light Gray
	Track Color	50% opacity of device color
Cards	Statistics Card	White background, 16px radius, bold value font (18-22px)
	Usage Card	Gradient Blue to Dark Blue, progress bar (White <80%, Orange >80%)
Navigation	Bottom Nav Bar	White background, 60px height, icons 22px, labels 11px
	Tabs	Home, Stats, Auto, Logs, Profile
	Selected Color	Blue
	Unselected Color	Gray
App Bar	Header	White background, no elevation, 22px Bold title, Dark Gray icons
Indicators	Status Badge	ON: Green (10% opacity), OFF: Red (10% opacity)
	Status Dot	ON: Green circle (8px), OFF: Gray circle (8px)
Notifications	Card	White (read) / Light Blue (unread), 16px radius, bold title, medium gray message text
Charts	Line Chart	Blue line 2.5px, white data points with blue stroke, gradient fill
	Donut Chart	Stroke width 14px, round caps, bold center text
Forms	Input Fields	White background, 12-16px radius, filled style, Blue prefix icon

Alerts	Snackbar	Success: Green, Error: Red, Warning: Orange, White text, 12px radius, 2-4s duration
Dialogs	Dialog Box	White background, 20px radius, Bold title (18-20px), Blue action buttons
Spacing	Screen Padding	14-16px; Between cards: 14-16px; Elements: 8-12px; Section margin: 16-20px
Gradients	Primary Gradient	Blue to Green diagonal, used for main buttons/hero sections
	Savings Card Gradient	Blue to Green diagonal, used for monthly savings card
	Header Gradient	Device color to lighter shade, used for device detail header

2.5.2. Software Interfaces

20. The system shall integrate with **Firebase Realtime Database** for device state synchronization.
21. **Firebase Authentication** shall handle user login and account management.
22. The mobile app shall be developed using **Flutter SDK (latest stable version)**.
23. ESP32 firmware shall be written in **Arduino C++**, using libraries for Wi-Fi and Firebase communication.
24. The system shall support **OTA (Over-The-Air) updates** for firmware maintenance.
25. No third-party commercial smart home ecosystems (e.g., Alexa, Google Home) are included in scope.

2.5.3. Hardware Interfaces

26. The ESP32 shall interface with a 4-channel relay module to control appliances (fan, light, curtain motor, water pump).
27. Sensors integrated include:
    - LM393 Fire Sensor (digital output to ESP32 GPIO).
    - MQ135 Smoke Sensor (analog output to ESP32 ADC).
    - PIR Motion Sensor (digital output).
    - LDR Light Sensor (analog output).
    - Water Level Sensor (digital/analog depending on configuration).
    - Temperature Sensor (e.g., DHT11/DHT22) for fan automation.
28. The buzzer shall be connected to ESP32 GPIO for alerts.
29. Communication between ESP32 and relay/sensors shall use standard GPIO pins with 3.3V logic.

2.5.4. Communications Interfaces

30. The ESP32 shall communicate with Firebase via **Wi-Fi (IEEE 802.11 b/g/n)**.
31. All data exchange shall use **HTTPS/TLS encryption**.

32. The mobile app shall synchronize appliance states and sensor data through Firebase in near real-time.
33. The system shall support **push notifications** for alerts (e.g. peak hour warnings).
34. No email or browser-based communication is required; all interactions occur through the mobile app.

2.6. Summary

The requirements analysis phase provided a structured foundation for the Smart Home Automation system. In this chapter, we identified the **user classes and their characteristics**, ensuring that the system design reflects the needs of residents, administrators, and external stakeholders. We then applied **requirement identification technique** such as use case modelling to capture functional expectations in a clear, verifiable manner.

Functional requirements were documented to specify the exact capabilities of the system, while **non-functional requirements** outlined quality attributes such as reliability, usability, performance, and security. These requirements ensure that the system is not only functional but also robust, user-friendly, and secure. Finally, the **external interface requirements** defined how the system interacts with users, software components, hardware devices, and communication protocols, ensuring seamless integration across all modules.

Overall, this chapter highlights the importance of the **requirements engineering process** in achieving project objectives. By systematically identifying, documenting, and validating requirements, we established a clear roadmap for design and implementation, reducing risks of scope creep and ensuring alignment with user expectations and project goals.

Chapter 3
Design and Architecture

3. System Design

The Smart Home Automation system operates in a **client–server architecture**, where the mobile application (client) communicates with the ESP32 microcontroller and Firebase cloud backend (server). The product perspective emphasizes seamless interaction between hardware sensors, actuators, and the mobile app interface. Dependencies include reliable Wi-Fi connectivity, Firebase services for authentication and real-time database synchronization, and compatibility with Android/iOS devices. Performance requirements such as low latency in device control and real-time sensor updates directly impact design decisions.

3.1. Design Considerations

Following design considerations were used.

3.1.1. Assumptions and Dependencies

35. Users will have access to a stable Wi-Fi connection for cloud synchronization.
36. The ESP32 microcontroller will remain the primary hardware platform for IoT integration.
37. Firebase services (Authentication, Realtime Database) will be available and maintained.
38. The mobile app will run on Android and iOS devices with minimum 720p resolution.
39. OTA (Over-The-Air) updates will be supported for firmware maintenance.

3.1.2. Limitations

40. Hardware scalability is limited to the number of GPIO pins available on ESP32.
41. System performance may degrade if multiple sensors and actuators are added beyond design capacity.
42. Dependency on third-party cloud services (Firebase) introduces external limitations.
43. Environmental factors (power outages, unstable internet) may affect system reliability.
44. The system currently does not integrate with commercial ecosystems (Alexa, Google Home).

3.1.3. Risks

45. **Connectivity Risk:** Loss of Wi-Fi may prevent real-time updates; mitigated by local fallback logic.
46. **Security Risk:** Unauthorized access attempts; mitigated by Firebase Authentication.
47. **Hardware Risk:** Sensor malfunction or relay failure; mitigated by modular design and replaceable components.
48. **Scalability Risk:** Increased load from multiple users/devices; mitigated by database structuring and modular expansion.

49. **Usability Risk:** Complex UI may reduce adoption; mitigated by adherence to Material Design guidelines and accessibility standards.

3.2. Design Models

To describe the system design of the Smart Home Automation project, we adopted an **object-oriented** development approach. The following models were developed to ensure clarity in system structure, interactions, and event handling.

3.2.1. Class Diagram

The class diagram illustrates the **static structure** of the system. It shows the major classes, their attributes, methods, and relationships.

- **Core classes:** FirebaseManager, SensorManager, Device, ModeController, SafetyMonitor, BuzzerController, UsageMonitor, BillingManager.
- **Relationships:**
  - SensorManager provides input to ModeController.
  - ModeController controls multiple Device objects.
  - Device, SafetyMonitor, UsageMonitor, and BillingManager all interact with FirebaseManager.
  - SafetyMonitor controls the BuzzerController.
- **Composition:** Attributes like RelayPin, Wattage, and State are intrinsic to Device; FireSensor and GasSensor are intrinsic to SafetyMonitor.

Figure *2* shows the class diagram.

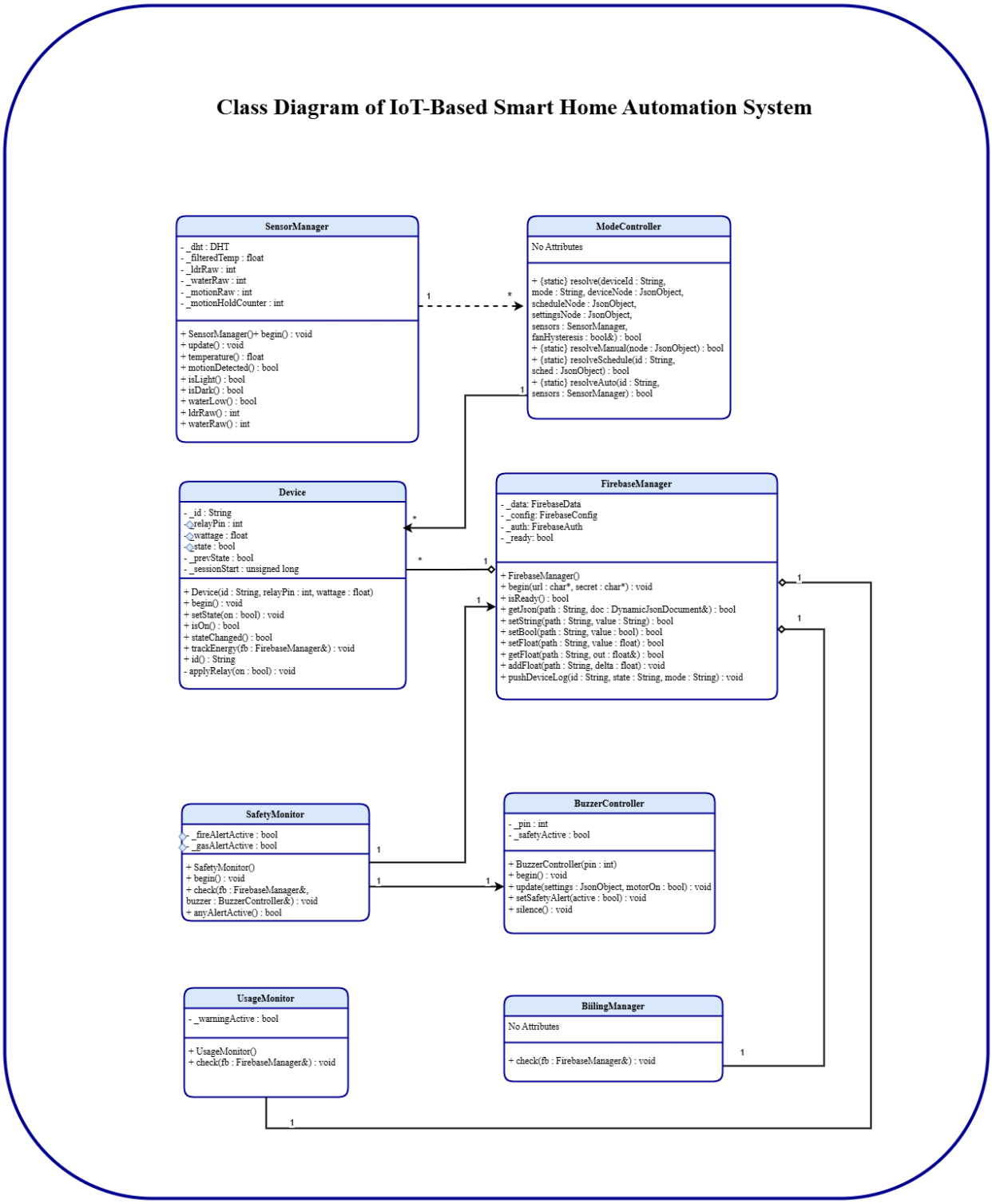


Figure 2: Class Diagram

3.2.2. Interaction Diagram (Sequence Diagram)

Interaction diagrams describe the dynamic behaviour of the system by showing how objects collaborate to achieve specific use cases. In this project, two sequence diagrams have been developed to illustrate the major workflows: **Device Control Interaction** and **Automation Execution.**

1. **Sequence Diagram — Device Control Interaction**

This diagram models the process of a user controlling a smart device through the mobile application.

- The User initiates the action by tapping ON/OFF in the Mobile App.
- The Mobile App sends a request to the FirebaseService, which updates the device state in the Firebase Realtime Database.
- The ESP32 Controller detects the change and calls Device::setState(newState) to update the device.
- The Relay Module executes digitalWrite(relayPin, HIGH/LOW) to physically switch the device.
- The Device confirms its new state with Device::isOn() and reports back.
- The ESP32 Controller pushes the updated status to Firebase using FirebaseManager::setBool("/devices/deviceId/state", actualState).
- Finally, the Mobile App retrieves the updated state and displays it to the user.

Figure *3* shows the sequence diagram for device control interaction.

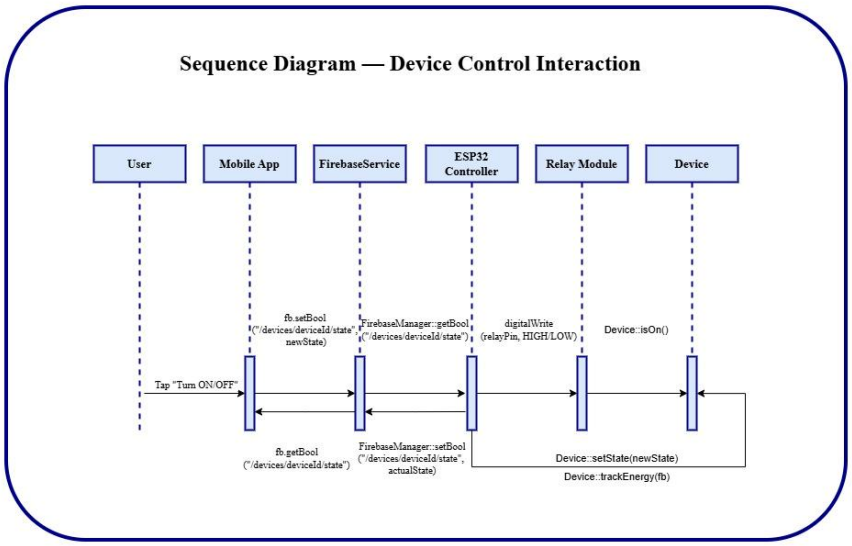


Figure 3: Sequence Diagram - Device Control Interaction

2. Sequence Diagram — Automation Execution

This diagram models the automatic execution of rules based on sensor input.

- A Sensor reads values using Sensor::readValue() and passes them to the ESP32 Controller.
- The ESP32 Controller invokes AutomationManager::applyRules(sensorId, value) and AutomationManager::evaluateConditions(sensorId, value) to check automation logic.
- If conditions are met, the controller triggers ESP32Controller::triggerAction(deviceId, newState).
- The Relay Module executes RelayController::activateRelay(deviceId, newState) to switch the device.
- The Device changes its state via Device::switchPower(newState) and reports Device::statusChanged(actualState).
- The ESP32 Controller logs usage with Device::trackEnergy(fb) and updates Firebase using FirebaseManager::updateDeviceState(deviceId, actualState).
- The Mobile App retrieves the updated state with MobileApp::updatedDeviceState(deviceId, actualState) and refreshes the UI.

Figure *4* shows the sequence diagram for automation execution.

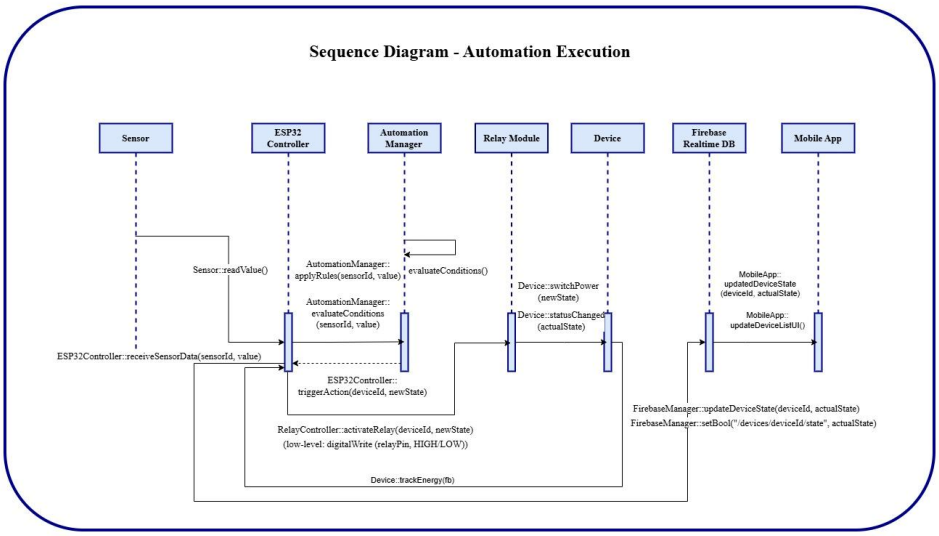


Figure 4: Sequence Diagram - Automation Execution

3.2.3. State Transition Diagram – Smart Light Control

The state transition diagram illustrates the event-driven behavior of the smart light subsystem. It shows how the light changes states in response to user actions and automation rules, ensuring both manual and automatic control are supported.

- Initial State → OFF: When the system starts (SystemStart), the light is initialized in the OFF state (Device::setState(OFF)).
- OFF → ON: Transition occurs when the user toggles ON through the mobile app. The ESP32 updates the relay (digitalWrite(relayPin, HIGH)), and the device switches power.
- ON → OFF: Transition occurs when the user toggles OFF. The ESP32 deactivates the relay (digitalWrite(relayPin, LOW)), and the device powers down.
- OFF/ON → AUTO: When automation mode is enabled (EnableAutoMode), control is delegated to the automation manager (AutomationManager::applyRules()).
- AUTO → ON: Automation rules or user override trigger the device ON state (Device::setState(ON)).
- AUTO → OFF: Automation rules or user override trigger the device OFF state (Device::setState(OFF)).

Figure *5* demonstrates how the system supports manual toggling, automation logic, and user overrides, ensuring flexibility and reliability in smart light control.

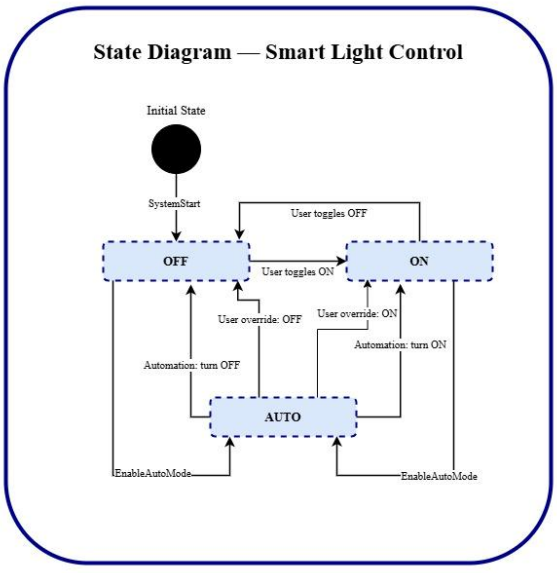


Figure 5: State Diagram - Smart Light Control

3.3. Architectural Design

The proposed system adopts a **three-tier client–server architecture**, ensuring clear separation of concerns, scalability, and maintainability.

- **Client Tier (Mobile Application):** Provides the user interface for device monitoring, manual control, automation configuration, scheduling, and peak hour customization. It also allows users to view usage logs and receive safety alerts.
- **Service Tier (Firebase Cloud Services):** Handles user authentication, real-time data synchronization, cloud storage, and communication between the mobile app and the ESP32 controller. It also stores device logs, usage data, and peak hour configurations.
- **Device Tier (ESP32 Controller + Modules):** Includes the ESP32 microcontroller, connected sensors (temperature, motion, light, water level, fire, smoke), and relay modules. This tier executes automation rules, responds to manual and scheduled commands, triggers safety alerts, and logs all device actions.

Figure *6* shows the smart home automation system architecture.

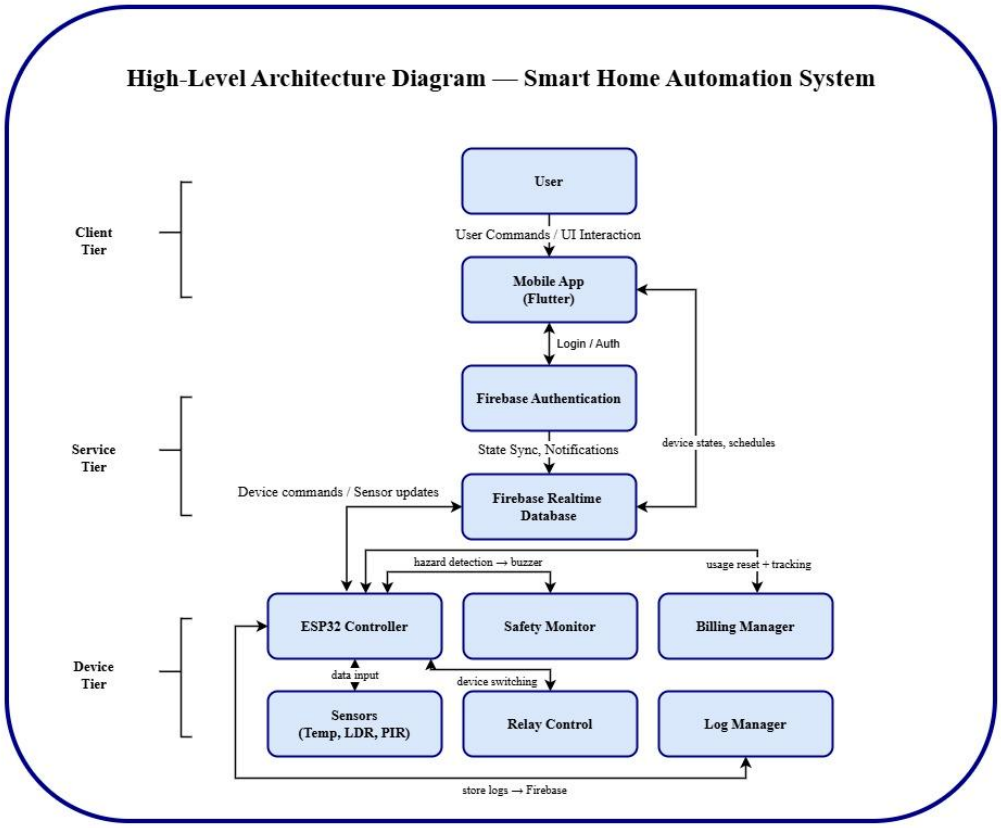


Figure 6: Architecture Diagram

3.3.1. UML Component diagram

The UML Component Diagram illustrates the modular structure of the smart home automation system.

Mobile App Component interacts with Firebase Authentication and Firebase Realtime Database for user login, device control, scheduling, peak hour configuration, and log retrieval.

Firebase Components include Authentication, Realtime Database, and Cloud Storage, which manage user accounts, device states, usage data, and logs.

ESP32 Controller Component communicates with:

- Sensor Interfaces (DHT11, PIR, LDR, Water Level, Fire, Smoke) for automation and safety monitoring.
- Relay Control for switching appliances ON/OFF.
- Safety Monitor for fire/smoke detection and buzzer alerts.
- Billing Manager for monthly usage resets and consumption tracking.
- Log Manager for recording device actions and syncing with Firebase.

Figure *7* shows the updated UML Component Diagram.

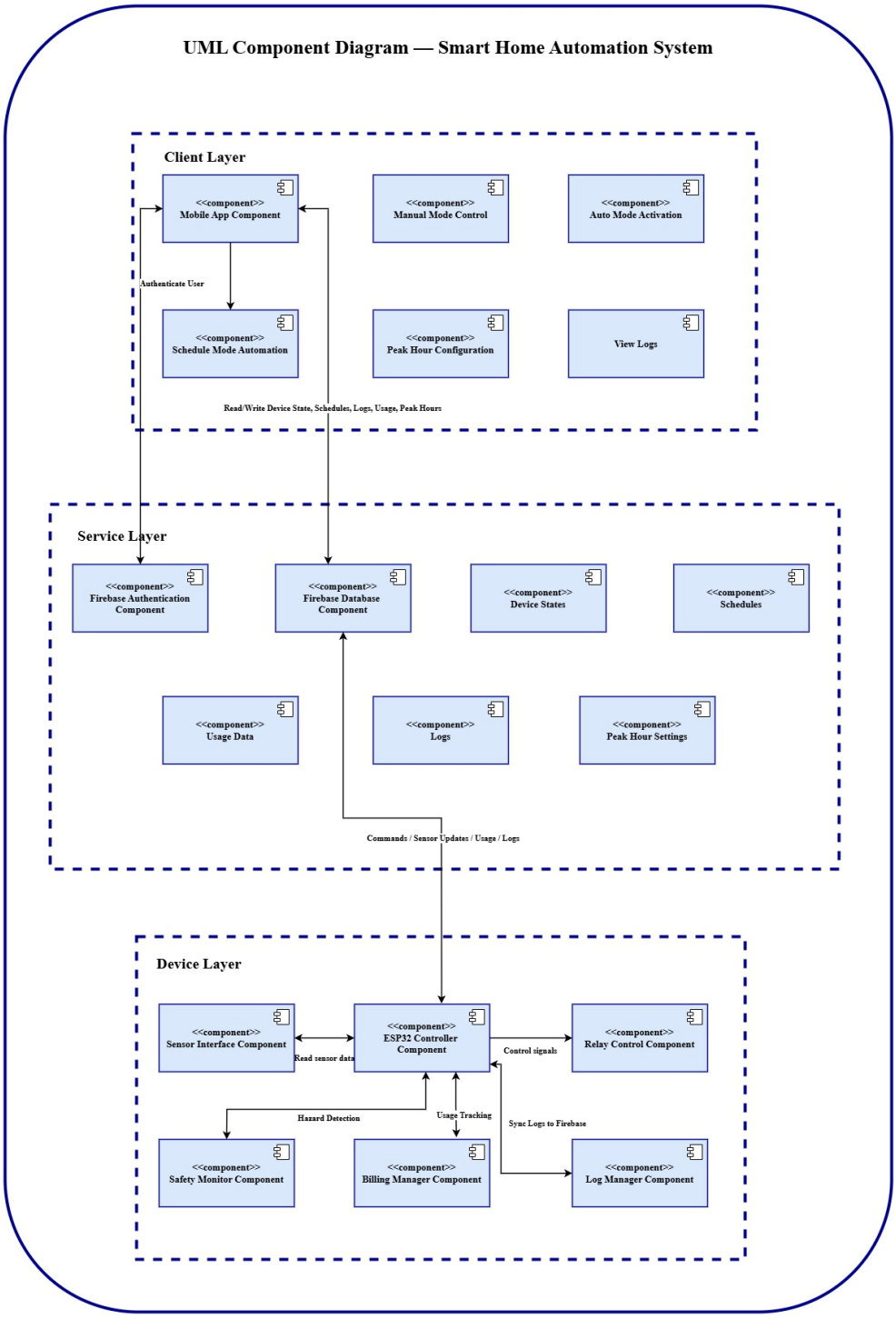


Figure 7: Component Diagram

3.4. Data Design

The smart home automation system transforms user commands, sensor inputs, and automation rules into structured data entities stored in Firebase Realtime Database. Each major entity (User, Device, Schedule, Log, Usage, Safety Alert) is represented with attributes and relationships that support system functionality.

3.4.1. Data Dictionary

Table *17* provides a comprehensive alphabetical listing of all major system entities, attributes, and parameters used within the Smart Home Automation System. Each entry specifies its data type and role, ensuring clarity on how information is stored, processed, and organized. This dictionary serves as a reference point for understanding the transformation of the information domain into structured data elements that underpin the system's functionality.

Table 17: Data Dictionary

Terminology	**Description**
ActionID	Integer — Unique identifier for each device action.
ActionType	Enum — Manual, Auto, Schedule, or Safety.
AlertID	Integer — Unique identifier for each safety alert.
AlertMessage	String — Description of hazard (e.g., Fire detected, Smoke detected).
BillingRecord	Object — Tracks monthly usage data (UserID, Month, Consumption, ResetFlag).
BuzzerPin	Integer — GPIO pin used to activate buzzer.
Consumption	Float — Energy usage in kWh tracked by Billing Manager.
ControllerID	String — Identifier for ESP32 controller instance.
DeviceID	String — Unique identifier for each appliance controlled.
DeviceState	Boolean — Current ON/OFF status of a device.
Email	String — User's registered email address.
LogEntry	Object — Contains LogID, DeviceID, ActionType, Timestamp.
LogID	Integer — Unique identifier for each log entry.
Mode	Enum — Current operating mode (Manual, Auto, Schedule).

Password	String — User's encrypted password (stored in Firebase Auth).
PeakHour	Object — Contains StartTime, EndTime for billing calculation.
RepeatFlag	Boolean — Indicates whether a schedule repeats daily/weekly.
Role	Enum — User roles (Admin, Standard User).
Schedule	Object — Contains ScheduleID, DeviceID, StartTime, EndTime, RepeatFlag.
ScheduleID	Integer — Unique identifier for each schedule.
SensorData	Object — Contains SensorID, Value, Timestamp.
SensorID	String — Identifier for each sensor (Temp, PIR, LDR, Fire, Smoke, Water).
StartTime	Time — Beginning of schedule or peak hour.
EndTime	Time — End of schedule or peak hour.
Timestamp	DateTime — Records when an event occurred.
UsageData	Float — Energy consumption tracked per device.
UserID	String — Unique identifier for each registered user.
UserProfile	Object — Contains UserID, Email, Password, Role.

3.4.2. Object-Oriented Approach

In addition to the flat alphabetical dictionary, the system design follows an Object-Oriented paradigm. The objects, their attributes, and associated methods are outlined below to demonstrate how responsibilities are encapsulated within distinct classes. This structured view highlights the relationships between components and clarifies how data entities interact through defined operations and parameters.

1. **User**

- Attributes: UserID, Email, Password, Role
- Methods: login(), configurePeakHour(start: Time, end: Time), viewLogs()

2. **Device**

    - Attributes: DeviceID, DeviceState
    - Methods: toggleDevice(state: Boolean)

3. **ESP32Controller**

    - Attributes: ControllerID, ConnectedDevices[]
    - Methods: processCommand(command: String), readSensorData(sensorID: String), executeSchedule(scheduleID: Integer)

4. **SafetyMonitor**

    - Attributes: AlertID, SensorData
    - Methods: triggerAlert(sensorID: String), activateBuzzer()

5. **BillingManager**

    - Attributes: UsageData, PeakHour
    - Methods: calculateUsage(), resetMonthly(), applyPeakHour(start: Time, end: Time)

6. **LogManager**

    - Attributes: LogEntry[]
    - Methods: storeLog(deviceID: String, actionType: Enum), syncLogs()

7. **ScheduleManager**

    - Attributes: Schedule[]
    - Methods: addSchedule(deviceID: String, start: Time, end: Time), removeSchedule(scheduleID: Integer), executeSchedule(scheduleID: Integer)

8. **BuzzerController**

    - Attributes: BuzzerPin
    - Methods: activateBuzzer(), deactivateBuzzer()

3.5. User Interface Design

The user interface of the Smart Home Automation System is designed to be simple, intuitive, and responsive, allowing users to control devices and monitor sensor data with ease. The mobile app follows Material Design guidelines to ensure a clean layout, clear visual hierarchy, and consistent interaction patterns. Users can securely log in, view all connected devices, check their current status, and control them with a single tap. The interface provides immediate visual feedback for every action through updated device states, and helpful status indicators, ensuring a smooth and reliable user experience. The interface layout and visual hierarchy were designed in alignment with the official Material Design guidelines to ensure accessibility and consistency (Team, 2024).

3.5.1. Screen Images

Below are some of the screens showing our app display.

3.5.1.1. Splash Screen

This screen appears when the application launches. It displays the app's branding theme and transitions into the authentication flow. **Figure 8** shows the splash screen.

Figure 8: Splash Screen

3.5.1.2. Dashboard Screen

The dashboard serves as the main navigation hub, allowing users to access devices, profile and overall system status. **Figure 9** shows dashboard screen.

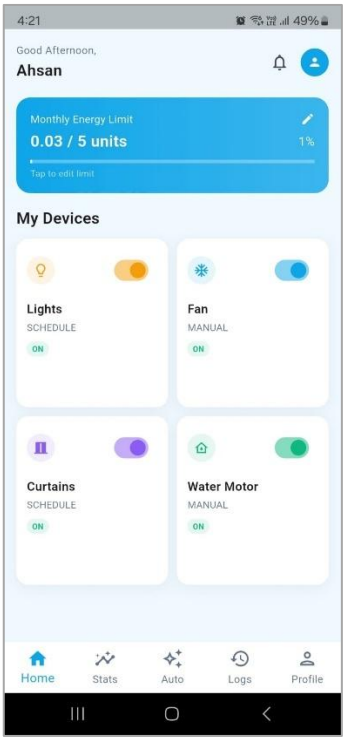


Figure 9: Dashboard Screen

3.5.1.3. Usage Stats Screen

The usage stats screen allows the user to view the monthly usage of the connected devices. **Figure *10*** shows dashboard screen.

Figure 10: Usage Screen

3.5.2. Screen Objects and Actions

This section describes the key screen objects and the actions associated with them for two representative use cases.

1. **Use Case 1: UC-3 Manual Mode (Device Control Screen)**
    - **Screen Objects:**
        - Toggle Button (ON/OFF)
        - Device Status Indicator (Green = ON, Red = OFF)
        - Feedback Label (“Device is now ON”)
    - **Actions:**
        - User taps the toggle → system sends command to Firebase → ESP32 updates relay → device switches ON/OFF.
        - Feedback is displayed instantly on the status indicator and label.

2. **Use Case 2: UC-5 Schedule Mode (Schedule Setup Screen)**

    - **Screen Objects:**

        - Time Picker (Start Time, End Time)
        - Device Dropdown (select appliance)
        - Save Button
        - Confirmation Message ("Schedule saved successfully")

    - **Actions:**

        - User selects device from dropdown → sets start/end times → taps Save.
        - System stores schedule in Firebase → ESP32 executes at specified times.
        - Confirmation message appears, and schedule is listed in the user's dashboard.

3.6. Behavioural Model

The Behavioural Model describes how the Smart Home Automation System operates during runtime, focusing on the flow of interactions between the user, mobile app, Firebase services, and the ESP32 hardware layer. State and sequence diagrams illustrate how devices change state and how components communicate during key operations.

3.6.1. Interaction Diagram (Sequence Diagram)

Interaction diagrams describe the dynamic behaviour of the system by showing how objects collaborate to achieve specific use cases. In this project, two sequence diagrams have been developed to illustrate the major workflows: **Device Control Interaction** and **Automation Execution.**

- **Sequence Diagram — Device Control Interaction**
  This diagram models the process of a user controlling a smart device through the mobile application.

    - The User initiates the action by tapping ON/OFF in the Mobile App.
    - The Mobile App sends a request to the FirebaseService, which updates the device state in the Firebase Realtime Database.
    - The ESP32 Controller detects the change and calls Device::setState(newState) to update the device.
    - The Relay Module executes digitalWrite(relayPin, HIGH/LOW) to physically switch the device.

- The Device confirms its new state with Device::isOn() and reports back.
- The ESP32 Controller pushes the updated status to Firebase using FirebaseManager::setBool("/devices/deviceId/state", actualState).
- Finally, the Mobile App retrieves the updated state and displays it to the user.

Figure *11* shows the sequence diagram for device control interaction.

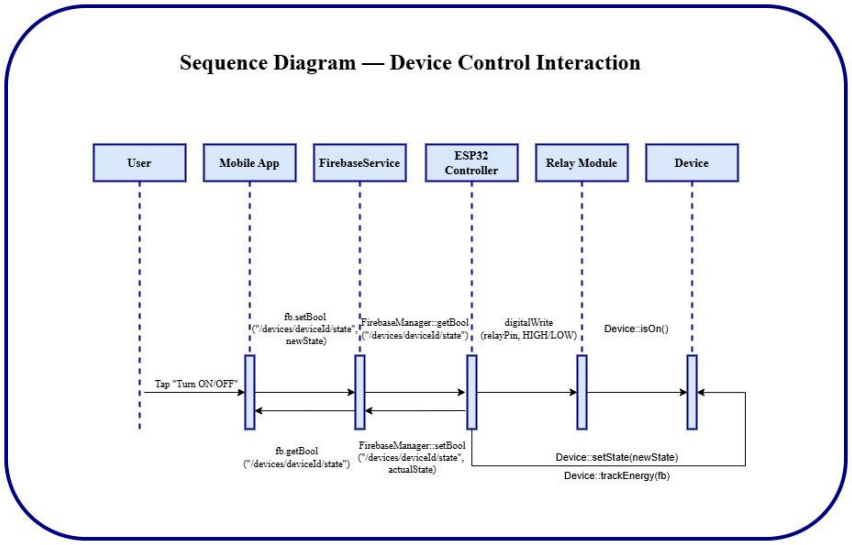


Figure 11: Sequence Diagram - Device Control Interaction

- **Sequence Diagram — Automation Execution**
  This diagram models the automatic execution of rules based on sensor input.

  - A Sensor reads values using Sensor::readValue() and passes them to the ESP32 Controller.
  - The ESP32 Controller invokes AutomationManager::applyRules(sensorId, value) and AutomationManager::evaluateConditions(sensorId, value) to check automation logic.
  - If conditions are met, the controller triggers ESP32Controller::triggerAction(deviceId, newState).
  - The Relay Module executes RelayController::activateRelay(deviceId, newState) to switch the device.
  - The Device changes its state via Device::switchPower(newState) and reports Device::statusChanged(actualState).

- The ESP32 Controller logs usage with Device::trackEnergy(fb) and updates Firebase using FirebaseManager::updateDeviceState(deviceId, actualState).
- The Mobile App retrieves the updated state with MobileApp::updatedDeviceState(deviceId, actualState) and refreshes the UI.

Figure *12* shows the sequence diagram for automation execution.

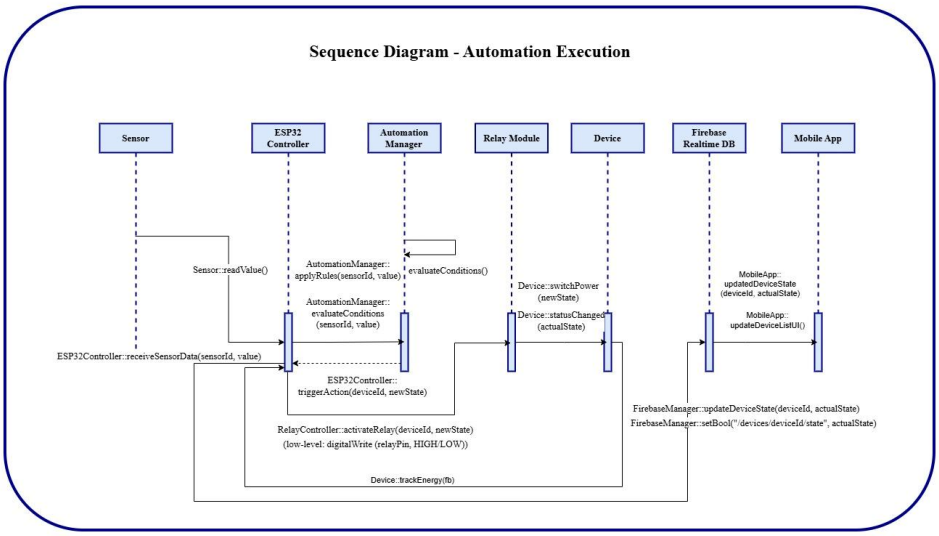


Figure 12: Sequence Diagram - Automation Execution

3.6.2. State Transition Diagram – Smart Light Control

The state transition diagram illustrates the event-driven behavior of the smart light subsystem. It shows how the light changes states in response to user actions and automation rules, ensuring both manual and automatic control are supported.

- Initial State → OFF: When the system starts (SystemStart), the light is initialized in the OFF state (Device::setState(OFF)).
- OFF → ON: Transition occurs when the user toggles ON through the mobile app. The ESP32 updates the relay (digitalWrite(relayPin, HIGH)), and the device switches power.
- ON → OFF: Transition occurs when the user toggles OFF. The ESP32 deactivates the relay (digitalWrite(relayPin, LOW)), and the device powers down.
- OFF/ON → AUTO: When automation mode is enabled (EnableAutoMode), control is delegated to the automation manager (AutomationManager::applyRules()).

- AUTO → ON: Automation rules or user override trigger the device ON state (Device::setState(ON)).
- AUTO → OFF: Automation rules or user override trigger the device OFF state (Device::setState(OFF)).

Figure *13* demonstrates how the system supports manual toggling, automation logic, and user overrides, ensuring flexibility and reliability in smart light control.

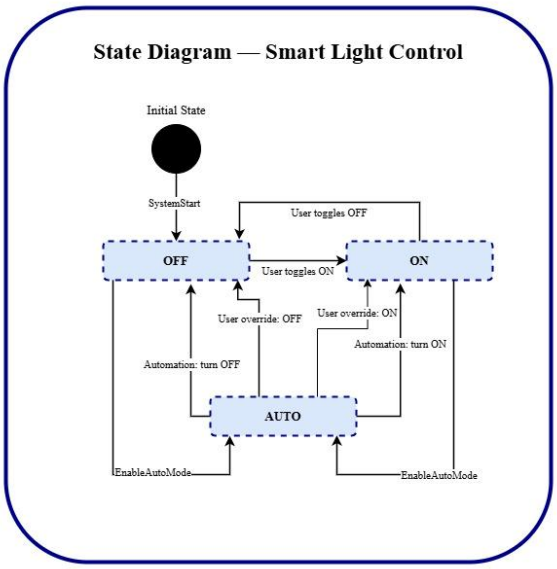


Figure 13: State Diagram - Smart Light Control

3.7. Design Decisions

This section outlines the major design choices made during the development of the IoT-Based Smart Home Automation System. Each decision was taken to ensure that the system remains scalable, maintainable, and capable of handling real-time operations.

- **Object-Oriented Design Pattern**

  The system was implemented using an Object-Oriented approach. This decision was made to encapsulate responsibilities within distinct classes (e.g., User, Device, SafetyMonitor, BillingManager), ensuring modularity, scalability, and ease of maintenance. OO design also supports future extensions without disrupting existing functionality.

- **Database Choice and Normalization**

  Firebase Realtime Database was selected as the backend due to its lightweight structure, real-time synchronization, and seamless integration with mobile

applications. A moderate level of normalization was applied to avoid redundancy while maintaining fast retrieval of device states, schedules, and logs.

- **Algorithmic Approaches**

  Event-driven triggers were used for safety monitoring (e.g., fire/smoke detection) to ensure immediate response. Time-based scheduling algorithms were adopted for automation, allowing users to configure start/end times with repeat flags. These approaches guarantee responsiveness and reliability in real-world scenarios.

- **User Interface Design**

  A mobile-first design was chosen, with clear typography, responsive layouts, and intuitive navigation. Feedback mechanisms such as status indicators, confirmation messages, and alert notifications were integrated to enhance usability and transparency.

3.8. Summary

This chapter outlined the key design ideas, models, and decisions that shaped the Smart Home Automation System. The Data Dictionary provided a structured view of system entities, while the Object-Oriented approach clarified how attributes and methods are organized across classes. Behavioural models illustrated dynamic workflows and lifecycle transitions. Design refinements, including modular OO principles, Firebase integration, and event-driven algorithms, were highlighted as deliberate choices to meet performance, usability, and scalability goals. Collectively, these decisions demonstrate how the design process directly supports the project's objectives of delivering a reliable, interactive, and efficient smart home application.

Chapter 4

Implementation

4. Implementation

This chapter discusses the implementation details of the Smart Home Automation System. The focus is on explaining the functionality of core components using pseudocode rather than source code. The pseudocode highlights the logical flow, data handling, and interaction between modules, while adhering to proper coding standards as outlined in Appendix-D.

4.1. Algorithm

This section presents the algorithms used in the Smart Home Automation System, along with their computational complexity and pseudocode representation. Each algorithm corresponds to a major module and demonstrates how the system achieves its core functionalities.

4.1.1. Algorithm 1: Device Control (Manual Mode)

- **Purpose:** Toggle device ON/OFF via mobile app.
- **Complexity:** O(1) - constant time operation.

Table 18: Algorithm 1 - Toggle Device

Algorithm ToggleDevice
Input: deviceID (String), state (Boolean)
Output: Confirmation message
1: CONNECT to Firebase 2: UPDATE deviceID.state = state 3: SEND command to ESP32 relay 4: DISPLAY feedback to user

4.1.2. Algorithm 2: Auto Mode (Sensor-Based Control)

- **Purpose:** Automatically control devices based on sensor input.
- **Complexity:** O(n) - depends on number of sensors checked.

Table 19: Algorithm 2 – Auto Mode

Algorithm AutoMode
Input: sensorData[]
Output: Device state updates
1: FOR each sensor in sensorData 2: IF motionDetected AND lightLevel < threshold 3: CALL ToggleDevice("Light", ON) 4: ELSE 5: CALL ToggleDevice("Light", OFF)

4.1.3. Algorithm 3: Safety Monitoring

- **Purpose:** Detect hazards (fire, smoke, water overflow) and trigger alerts.
- **Complexity:** O(n) — checks all safety sensors sequentially.

Table 20: Algorithm 3 – Safety Monitoring

Algorithm TriggerAlert
Input: sensorID (String), sensorValue (Float)
Output: Alert notification
1: IF sensorValue > threshold 2: CALL BuzzerController.activateBuzzer() 3: SEND alert notification to user

4.1.4. Algorithm 4: Billing Calculation

- **Purpose:** Calculate monthly energy usage.
- **Complexity:** O(n) — iterates over all devices.

Table 21: Algorithm 4 – Billing Calculation

Algorithm CalculateUsage
Input: DeviceList[]
Output: MonthlyUsage (Float)
1: MonthlyUsage ← 0 2: FOR each device in DeviceList 3: MonthlyUsage += device.powerConsumption * timeActive 4: RETURN MonthlyUsage

4.2. External APIs/SDKs

This section describes the third-party APIs and SDKs used in the project implementation. Each entry specifies the API name and version, its description, purpose of usage, and the specific endpoints, functions, or classes where it is applied as shown in **Table *22***.

Table 22: External APIs and SDK

Name of API / SDK	Description	Purpose of Usage	Endpoints / Functions / Classes
Firebase Realtime Database (v9.0)	Cloud-hosted NoSQL database with real-time synchronization	Store and retrieve device states, schedules, and logs	FirebaseManager.writeDeviceState(), FirebaseManager.readSchedule(), FirebaseService.syncData()
Firebase Authentication (v9.0)	Provides secure user authentication via email/password or tokens	Manage user login and registration	UserAuth.login(), UserAuth.register(), FirebaseAuth.instance.currentUser
Flutter SDK (v3.41.9)	Cross-platform UI toolkit for building mobile apps	Develop mobile app interface and bind UI actions to backend	LoginScreen, DashboardScreen, DeviceControl, ScheduleScreen
Arduino IDE (v2.3.8)	Open-source development environment for microcontrollers	Program ESP32 with C/C++ firmware for device control	setup(), loop(), digitalWrite(), analogRead()

4.3. Code Repository

In order to manage version control and collaboration effectively for the project, Git was used as the primary tool. All project files, including source code, documentation, and other relevant resources, were stored in the Git repository. This ensured proper tracking of changes, collaboration among contributors, and access to the most up-to-date version of the project.

Git Repository Link

The repository for the group project can be accessed using the following link:
https://github.com/Itx-Mohsin-Dev/smart-home-pro.git

4.3.1. Metrics of the Git Repository

The following metrics will be monitored to ensure effective use of the Git repository:

- **Commits**: The number of commits made to track the frequency are 4.
- **Branches**: The number of active and merged branches are 2.
- **Issues**: The number of open and closed issues, indicating bug tracking, feature requests, or task management are 0.
- **Contributors**: A list of contributors and their contribution statistics (commits, lines added, and lines removed).
- **Code Reviews**: The number of reviews conducted on pull requests to ensure code quality and compliance with project standards.

4.4. Summary

This chapter presented the implementation details of the Smart Home Automation System. The core functionalities were explained through pseudocode, highlighting the logical flow of modules such as device control, auto mode, safety monitoring, and billing calculation. Algorithms were described along with their computational complexities, demonstrating efficiency and scalability in handling real-time operations. External APIs and SDKs, including Firebase, Flutter, and Arduino IDE, were documented to show how third-party tools supported seamless integration between hardware and software. The Git repository setup was also discussed, emphasizing version control, collaboration, and quality assurance through metrics such as commits, branches, pull requests, issues, contributors, and code reviews.

By bridging design decisions with implementation details, this chapter reinforces the project's objectives of delivering a reliable, interactive, and efficient smart home solution. It demonstrates how theoretical models were translated into practical modules, ensuring that the system is both technically sound and user-oriented.

Chapter 5
Testing and Evaluation

5. Introduction

In order to ensure software verification and validation, systematic testing was conducted for the Smart Home Automation System. Testing helps confirm that the implemented modules meet the specified requirements and function correctly under different scenarios. While automated test scripts are preferred for efficiency and repeatability, manual test cases were also designed to validate core features and edge conditions. All test scripts and test cases conform to best practices, ensuring reliability and clarity in the testing process.

Best Practices Followed

- **Keep Tests Independent:** Each test case was designed to run independently without relying on the outcome of other tests.
- **Use Meaningful Assertions:** Assertions were written to clearly validate expected outcomes.
- **Handle Test Data:** Setup and teardown methods were used to prepare and reset test data.
- **Avoid Hardcoding:** Variables and configuration files were used instead of hardcoded values.
- **Make Tests Repeatable:** Tests were structured to run multiple times without failure due to state persistence.

5.1. Unit Testing (UT)

Unit testing was carried out to validate the correctness of individual modules within the Smart Home Automation System. Each unit, such as device control, billing calculation, and safety monitoring, was tested in isolation to ensure that its logic and outputs matched the design specifications. By keeping tests independent and repeatable, unit testing confirmed that the building blocks of the system were reliable before integration.

- **Remote Appliance Control (FR-4)**

Following is the test case for device control as shown in **Table *23***.

Table 23: Unit Testing - Remote Appliance Control

Field	**Entry**
Testcase ID	UT1
Requirement ID	FR-4
Title	Remote Appliance ON/OFF Control
Description	Verify that the user can toggle appliances ON/OFF via the mobile app.

Objective	Ensure Firebase updates relay state correctly.
Driver/Precondition	ESP32 connected, Firebase initialized.
Test Steps	1. Toggle device ON in app 2. Observe relay pin state 3. Check Firebase entry
Input	Device = Light, State = ON
Expected Results	Relay pin HIGH, Firebase updated to ON
Actual Result	Relay pin HIGH, Firebase updated to ON
Remarks	Pass

50. Billing Manager (FR-9)

Following is the test case for billing manager as shown in **Table *24***.

Table 24: Unit Testing – Billing Manager

Field	**Entry**
Testcase ID	UT2
Requirement ID	FR-9
Title	Water Pump Usage Calculation
Description	Verify that pump usage is calculated correctly based on water level sensor input.
Objective	Ensure calculateUsage() returns correct values.
Driver/Precondition	Pump active for 2 hours, sensor thresholds defined.
Test Steps	1. Simulate water level 2. Run usage calculation 3. Observe returned value
Input	Pump = ON, Duration = 2h
Expected Results	Usage logged correctly
Actual Result	Usage logged correctly

Remarks	Pass

5.2. Functional Testing

Functional testing focused on verifying that the implemented modules met stakeholder requirements and system specifications. This level of testing ensured that the mobile app, automation features, and authentication mechanisms behaved as expected from the user's perspective. Functional tests validated end-to-end workflows, confirming that the system delivered the intended functionality in real-world scenarios.

- **Automatic Mode Activation (FR-5)**

Following is the test case for auto mode as shown in **Table *25***.

Table 25: Functional Testing – Auto Mode

Field	**Entry**
Testcase ID	FT1
Requirement ID	FR-5
Title	Automatic Mode Activation
Description	Verify that ESP32 takes control of appliances when Auto Mode is enabled.
Objective	Ensure Auto Mode overrides manual commands.
Driver/Precondition	Auto Mode enabled in app.
Test Steps	1. Enable Auto Mode 2. Simulate sensor input 3. Observe appliance state
Input	Motion = TRUE, Temp = 32°C
Expected Results	Fan ON, Light ON
Actual Result	Fan ON, Light ON
Remarks	Pass

- **User Login (FR-2)**

Following is the test case for user login as shown in **Table *26***.

Table 26: Functional Testing – User Login

Field	**Entry**
Testcase ID	FT2
Requirement ID	FR-2
Title	User Login
Description	Verify that registered users can log in with valid credentials.
Objective	Ensure Firebase Authentication grants access.
Driver/Precondition	User account exists in Firebase.
Test Steps	1. Enter valid email/password 2. Click login 3. Observe dashboard access
Input	Email = test@example.com, Password = 123456
Expected Results	User logged in, dashboard displayed
Actual Result	User logged in, dashboard displayed
Remarks	Pass

5.3. Integration Testing (IT)

Integration testing was conducted after successful unit and functional tests to evaluate the interaction between modules. The purpose was to ensure that components such as ESP32 firmware, Firebase backend, and Flutter frontend worked together seamlessly. By simulating real data flows across modules, integration testing confirmed that combined features yielded the desired results without conflicts or errors.

- **Fire and Smoke Detection (FR-13)**

Following is the test case for smoke detection as shown in **Table *27***.

Table 27: Integration Testing – Fire and Smoke Detection

Field	Entry
Testcase ID	IT1
Requirement ID	FR-13
Title	Fire and Smoke Detection
Description	Verify that fire/smoke sensors trigger buzzer and app alert.
Objective	Ensure integration between ESP32, Firebase, and Flutter app.
Driver/Precondition	Fire and smoke sensors connected, app logged in.
Test Steps	1. Simulate fire sensor value above threshold 2. Observe buzzer state 3. Check app notification
Input	FireSensor = 120, SmokeSensor = abnormal
Expected Results	Buzzer ON, app notification received
Actual Result	Buzzer ON, app notification received
Remarks	Pass

- **Schedule Mode (FR-12)**

  Following is the test case for schedule mode as shown in **Table 28**.

Table 28: Integration Testing – Schedule Mode

Field	Entry
Testcase ID	IT2
Requirement ID	FR-12
Title	Scheduled Device Activation
Description	Verify scheduled task triggers device ON at set time.

Objective	Ensure Firebase schedule integrates with ESP32.
Driver/Precondition	Schedule set for 10:00 AM.
Test Steps	1. Create schedule in app 2. Wait until scheduled time 3. Observe device state
Input	Device = Light, Time = 10:00 AM
Expected Results	Light turns ON at 10:00 AM
Actual Result	Light turns ON at 10:00 AM
Remarks	Pass

5.4. Performance Testing

Performance testing was performed to assess the responsiveness and stability of the system under varying loads. The focus was on measuring device toggle latency, synchronization speed with Firebase, and system reliability under stress conditions. Tools such as Firebase Performance Monitoring and Flutter DevTools were used to capture metrics, and screenshots of configurations and outputs were documented to demonstrate system efficiency.

- **Response Time (NFR-3)**
  Following is the test case for response time as shown in **Table *29***.

Table 29: Performance Testing - Response Time

Field	**Entry**
Testcase ID	PT1
Requirement ID	NFR-3
Title	Device Toggle Latency
Description	Measure time taken to toggle device via app.
Objective	Ensure system responds within acceptable latency.
Driver/Precondition	Stable Wi-Fi connection, Firebase initialized.

Test Steps	1. Toggle device ON from app 2. Measure time until relay activates
Input	Device = Fan, State = ON
Expected Results	Response time < =3 second
Actual Result	Response time = 2.7 seconds
Remarks	Pass

- **Reliability Under Stress (NFR-1)**
  Following is the test case for reliability as shown in **Table *30***.

Table 30: Performance Testing - Reliability

Field	**Entry**
Testcase ID	PT2
Requirement ID	NFR-1
Title	Stress Test with Multiple Requests
Description	Verify system stability when multiple toggle requests are sent quickly.
Objective	Ensure system does not crash under stress.
Driver/Precondition	Wi-Fi connected; Firebase active.
Test Steps	1. Send 50 toggle requests in 1 minute 2. Observe system behaviour
Input	Device = Light, State = ON/OFF repeatedly
Expected Results	All requests processed, no crash
Actual Result	All requests processed, no crash
Remarks	Pass

5.5. Summary

This chapter outlined the verification and validation process applied to the Smart Home Automation System. Unit testing confirmed the correctness of individual modules such as appliance control and billing calculations. Functional testing validated that the system met stakeholder requirements, including user authentication, automatic mode activation, and device scheduling. Integration testing ensured seamless communication between ESP32 hardware, Firebase backend, and the Flutter mobile application, with successful validation of safety features like fire and smoke detection. Performance testing assessed response times and system stability under stress, demonstrating that the system maintained reliability and efficiency even under heavy load.

By covering unit, functional, integration, and performance testing, this chapter highlights the comprehensive approach taken to ensure system quality. The documented test cases provide traceability to both functional and non-functional requirements, reinforcing that the project objectives of reliability, usability, automation, and safety were achieved. This testing process establishes confidence in the system's readiness for deployment and its ability to deliver a secure, efficient, and user-friendly smart home solution.

Chapter 6
System Conversion

6. Introduction

System conversion refers to the process of transitioning from an old information system to a new one. It involves transferring data, processes, and operations to ensure that the new system can fully replace the old environment. The choice of conversion method is critical, as it determines the risk, cost, and reliability of the transition. For the Smart Home Automation System, conversion ensures that users can seamlessly adopt the new application without disruption to their daily routines.

6.1. Conversion Method

For the Smart Home Automation System, the **Pilot Conversion approach** was chosen. In this method, the new system is first implemented in a limited scope - for example, within a single household or controlled lab environment - before being rolled out to all users. This allows the project team to validate functionality, performance, and reliability under real-world conditions while minimizing risk. By restricting the initial deployment, any issues can be identified and corrected without affecting the entire user base.

The pilot conversion method is particularly suitable for this project because smart home systems directly impact daily routines and safety. A sudden direct conversion could expose users to risks if errors occur, while parallel conversion would be unnecessarily costly and complex. Phased conversion is useful for large enterprise systems, but in this case, a pilot deployment provides the best balance of safety, cost, and efficiency. Once the pilot environment demonstrates stability and user satisfaction, the system can be scaled to additional households in phases, ensuring smooth adoption.

6.2. Deployment

Deployment is the process of building the application, configuring the environment, and making the system operational for end users. Since the Smart Home Automation mobile app is developed in Flutter and integrated with Firebase, it cannot be containerized using Docker. Instead, the deployment strategy involves generating an APK file for Android devices, installing it on smartphones, and ensuring Firebase services are properly configured. This ensures that users can directly interact with the system through the mobile application while ESP32 devices communicate with Firebase in real time.

Deployment Steps:

1. **Build the APK:**

    - Use Flutter build tools (flutter build apk) to generate the release version.
    - Sign the APK with a release key for secure distribution.

2. **Firebase Configuration:**

    - Ensure Firebase Authentication and Realtime Database are active.
    - Upload configuration file (google-services.json) into the Flutter project.

3. **Database Restore:**

    - Restore Firebase Realtime Database from backup (JSON export).
    - Validate that device states, schedules, and user accounts are correctly loaded.

4. **Installation:**

    - Distribute APK to users.
    - Install on Android devices.

5. **Operational Check:**

    - Log in with a test account.
    - Toggle appliances ON/OFF.
    - Validate synchronization between app, Firebase, and ESP32.

6.2.1. Data Conversion

Data conversion ensures that existing information is properly migrated into the deployed system. For this project, data conversion primarily involves Firebase Realtime Database setup and restoration.

Steps:

- Extract existing device states and schedules from the development database.
- Clean and validate data (remove test accounts, invalid entries).
- Load validated data into Firebase Realtime Database.
- Perform backup and restore tests to confirm recovery procedures.

This guarantees that the deployed system starts with accurate, reliable data and can recover from failures if needed.

6.2.2. Training

Training ensures that users can operate the system effectively. A concise user manual was prepared to guide users through the two most important use cases:

Use Case 1: User Login & Registration

- Open the mobile app.
- Register with a valid email and password (minimum 8 characters, one letter, one number).

- Log in using credentials.
- Dashboard loads with connected devices.

Use Case 2: Appliance Control & Automation

- Select a device (Fan, Light, Curtain Motor, Water Pump).
- Toggle ON/OFF manually.
- Enable Automatic Mode to allow ESP32 to control devices based on sensor inputs.
- Verify status updates in real time on the app.

6.3. Post-Deployment Training

Once the system is deployed, post-deployment testing is conducted to ensure that the application is operational and that data conversion has been successful. This involves rerunning the test cases documented in Chapter 5 to validate that the deployed APK, Firebase backend, and ESP32 devices are functioning correctly. Key checks include verifying user login, appliance control, automatic mode activation, schedule execution, and safety alerts. Performance tests are also repeated to confirm that response times and reliability remain within acceptable limits. Successful completion of these tests demonstrates that the system has been properly deployed and converted.

6.4. Challenges

During deployment, several challenges were encountered:

1. **Firebase Backup and Restore**
    - **Challenge:** Ensuring that both schema and live data were backed up and restored correctly.
    - **Solution:** Used Firebase Console's JSON export/import feature and validated restored data against test accounts.

2. **APK Distribution**
    - **Challenge**: Since Docker deployment was not feasible, distributing the APK securely was necessary.
    - **Solution:** Shared the apk of app on different communication channels to test users.

3. **Device Connectivity**
    - **Challenge:** ESP32 devices occasionally failed to sync with Firebase after redeployment.
    - **Solution:** Added retry logic and verified Wi-Fi stability before operational rollout.

4. **User Training**
    - **Challenge:** New users required guidance on login and automation features.

- **Solution:** Developed a concise user manual with screenshots of the two main use cases.

6.5. Summary

This chapter presented the deployment process for the Smart Home Automation System. The APK build and Firebase configuration steps were outlined, along with data conversion procedures and user training. Post-deployment testing confirmed that the system was properly deployed and operational. Challenges such as Firebase backup, APK distribution, device connectivity, and user training were encountered, but each was resolved through structured solutions. Overall, the deployment process ensured that the system met its objectives of reliability, usability, and safety, and is now ready for operational usage.

Chapter 7
Conclusion

7. Introduction

This chapter concludes the Final Year Design Project (FYDP) by evaluating the objectives and goals specified in Chapter 1. It traces how each objective was implemented in the developed Smart Home Automation System and highlights areas for future improvement. The evaluation demonstrates that the project successfully met its core objectives of automation, safety, usability, and reliability, while also identifying opportunities for scalability and enhancement.

7.1. Evaluation

Table *31* lists the objectives defined in Chapter 1 and their implementation status in the developed system:

Table 31: Evaluation

Objective	Status
Provide secure user login and authentication via Firebase	Completed
Enable remote ON/OFF control of appliances (Fan, Light, Curtain Motor, Water Pump)	Completed
Support automation modes (Manual, Auto, Schedule)	Completed
Implement temperature-based fan control for energy efficiency	Completed
Automate lighting based on motion detection	Completed
Automate curtain movement based on ambient light levels	Completed
Automate water pump operation based on tank water levels	Completed
Provide buzzer alerts during peak electricity hours	Completed
Integrate fire and smoke detection sensors with buzzer alerts	Completed
Ensure secure communication between mobile app, Firebase, and ESP32	Completed
Provide scheduling functionality for appliances	Completed
Achieve performance goals (response time = 3 seconds, stable under stress)	Completed
Ensure usability (easy learning within 10 minutes, minimal interactions)	Completed
Maintain affordability using low-cost hardware	Completed
Support portability (cross-platform Flutter app)	Completed

Allow scalability for additional sensors and devices	Partially Completed – Future Work

7.2. Traceability Matrix

The traceability matrix ensures that every requirement defined in the Software Requirements Specification (SRS) is accounted for in the design, code, and testing phases. This provides confidence that the final product fully implements the intended functionality.

Table 32: Req. Traceability Matrix

Req. ID	Req. Description	Design Specification	Code (Project)	Test ID
FR-1	User Registration	Authentication Component	lib/services/auth_service.dart (Flutter)	FT2
FR-2	User Login	Authentication Component	lib/services/auth_service.dart (Flutter)	FT2
FR-3	Device Onboarding	Device Manager Class	lib/services/device_manager.dart (Flutter) + main.cpp (ESP32)	IT2
FR-4	Remote Appliance Control	Relay Control Module	main.cpp (ESP32 relay logic) + lib/services/firebase_service.dart	UT1
FR-5	Automatic Mode Activation	Automation Controller	main.cpp (ESP32 automation loop)	FT1
FR-6	Temperature Based Fan Control	Sensor Handler	main.cpp (DHT sensor logic)	FT3
FR-7	Motion Based Light Automation	PIR Sensor Module	main.cpp (PIR sensor interrupt)	FT4
FR-8	Light Level Curtain Automation	LDR Sensor Module	main.cpp (LDR sensor logic)	FT5
FR-9	Water Level Pump Automation	Water Sensor Module	main.cpp (Water level sensor logic)	UT2
FR-10	Peak Time Buzzer Alert	Buzzer Controller	main.cpp (Buzzer logic)	IT3
FR-11	Communication Handling	Firebase Sync Service	lib/services/firebase_service.dart (Flutter) + main.cpp (ESP32 Wi-Fi)	PT1

FR-12	Schedule Mode	Scheduler Component	lib/services/scheduler.dart (Flutter) + main.cpp (ESP32 task execution)	IT2
FR-13	Fire and Smoke Detection	Safety Module	main.cpp (LM393 + MQ135 sensor logic)	IT1
NFR-1	Reliability	System Architecture	main.cpp (ESP32 watchdog + retries)	PT2
NFR-2	Usability	Flutter UI Components	lib/ui/dashboard.dart	FT2
NFR-3	Performance	Firebase Sync Service	lib/services/firebase_service.dart	PT1
NFR-4	Security	Firebase Auth Config	lib/services/auth_service.dart	FT2
NFR-5	Scalability	Modular Architecture	lib/services/device_manager.dart	(Future Test)
NFR-6	Portability	Flutter Cross-Platform	main.dart (Flutter entry point)	FT2
NFR-7	Affordability	Hardware Design	ESP32 + Sensors (LM393, MQ135, PIR, LDR, Water Level)	(Not Applicable)

7.3. Conclusion

This project successfully delivered a Smart Home Automation System integrating ESP32 hardware, Firebase backend, and a Flutter mobile application. The system achieved secure authentication, remote appliance control, automation through sensor inputs, scheduling, and safety monitoring. By implementing all functional and non-functional requirements, the project demonstrated reliability, usability, affordability, and portability. The contributions are significant because they provide a low-cost, scalable, and user-friendly smart home solution that enhances convenience, energy efficiency, and safety.

Table 33: Objectives and Functionality

Objectives	**Functionality Provided**
Secure user login and authentication	Firebase Authentication implemented in auth_service.dart
Remote ON/OFF control of appliances	Relay control via ESP32 and Firebase commands
Automation modes (Manual, Auto, Schedule)	Implemented in automation_controller.dart and scheduler.dart

Temperature-based fan control	DHT sensor logic in ESP32 firmware
Motion-based light automation	PIR sensor integration for occupancy detection
Curtain automation based on light levels	LDR sensor logic controlling curtain motor
Water pump automation based on tank levels	Water level sensor logic in ESP32
Peak time buzzer alerts	Buzzer controller logic in ESP32
Fire and smoke detection	LM393 + MQ135 sensor integration with buzzer alerts
Secure communication	Firebase Realtime Database with HTTPS/TLS
Scheduling functionality	Scheduler component in Flutter + ESP32 task execution
Reliability and performance	Response time <= 3s, stable under stress
Usability	Flutter UI dashboard, intuitive controls
Affordability	ESP32 + low-cost sensors under PKR 10,000
Portability	Flutter APK for Android, cross-platform support
Scalability	Modular design, support for additional sensors (future work)

7.4. Future Work

While the current system meets its defined objectives, several enhancements can be considered for future development:

- **Scalability:** Extend support for additional sensors (humidity, CO_2, gas) and more appliances.
- **Cloud Migration:** Explore deployment on enterprise IoT platforms (AWS IoT, Google Cloud IoT) for larger-scale usage.
- **Advanced Analytics:** Integrate dashboards for energy usage trends, predictive maintenance, and anomaly detection.
- **Voice Control:** Add integration with Google Assistant or Alexa for improved usability.
- **Enhanced Security:** Implement multi-factor authentication and role-based access control.
- **Cross-Platform Distribution:** Package the app for iOS alongside Android APK to widen accessibility.

References

- Ahmad, S. K. (2023). Cloud-Based Smart Home Management System for Energy Optimization. *Springer Nature Computer Science, 4*(3), 90-101.
- Al-Fuqaha, A. G. (2015). A Survey on Enabling Technologies, Protocols, and Applications. *IEEE Communications Surveys & Tutorials, 17*(4), 2347–2376.
- Al-Fuqaha, A., Guibene, M., & Mohamed, N. (2015). Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications. *IEEE Communications Surveys & Tutorials, 17*(4), 2347–2376.
- Al-Quraishi, M. M.-R. (2021). AI-Powered Smart Home Automation: Integrating IoT with Machine Learning for Energy Efficiency. *IEEE Access, 9*, 116234–116245.
- Arduino. (2024). *Arduino Official Documentation*. Retrieved November 10, 2024 from https://docs.arduino.cc/
- Association, I. S. (2025). *IEEE P2890—Standard for Smart Home Energy Efficiency and Interoperability.*
- Chen, M., Wang, L., & Zhang, H. (2025). Edge-AI for Real-Time IoT Automation: Reducing Latency in Smart Home Systems. *IEEE Internet of Things Journal*, 4500–4515.
- Developers, G. (2024, Novemberx 10). *Flutter Documentation*. From https://docs.flutter.dev/
- Firebase, G. (2024). *Firebase Documentation*. Retrieved November 10, 2024 from https://firebase.google.com/docs
- Kaur, G. &. (2020). A Review on Smart Home Automation Techniques Using IoT. *Journal of Ambient Intelligence and Humanized Computing, 11*(8), 3465–3478.
- Khan, A. R., & Singh, S. P. (2025). Federated Learning for Privacy-Preserving Smart Home Energy Management. *ACM Transactions on Internet of Things*, 112–130.
- Nguyen, T., Patel, B., & Kumar, R. (2025). Low-Cost Arduino-Based Smart Plugs with Cloud-Integrated Energy Monitoring. *Proc. IEEE International Conference on Consumer Electronics (ICCE)*, (pp. 1–6). Las Vegas.
- Sharma, R. &. (2022). IoT-Based Smart Home Automation System Using Arduino and Mobile Application. *International Journal of Computer Applications, 183*(22), 15–20.
- Society, I. C. (1998). *IEEE Recommended Practice for Software Requirements Specifications.* New York.

- Sommerville, I. (2016). *Software Engineering.* Boston: Pearson Education.
- Sommerville, I. (2016). *Software Engineering, 10th Edition.* Harlow: Pearson Education.
- Systems, E. (2024). *ESP8266/ESP32 Technical Reference*. From https://www.espressif.com/en/support/documents/technical-documents
- Team, G. M. (2024). *Material Design Guidelines*. From https://m3.material.io
- TensorFlow. (2024). *TensorFlow Lite for Microcontrollers*. Retrieved November 10, 2024 from https://www.tensorflow.org/lite/microcontrollers

Appendix A

Use case Description Template

Appendix-A Use Case Description (Fully Dressed Format)

Table *34* below indicates a comprehensive use case template

Table 34: Use Case Format

Use Case ID:	Enter a unique numeric identifier for the Use Case. e.g. UC-1
Use Case Name:	Enter a short name for the Use Case using an active verb phrase.
Actors:	[An actor is a person or other entity external to the software system being specified who interacts with the system and performs use cases to accomplish tasks.]
Description:	[Provide a brief description of the reason for and outcome of this use case.]
Trigger:	[Identify the event that initiates the use case.]
Preconditions:	[List any activities that must take place, or any conditions that must be true, before the use case can be started.]
Postconditions:	[Describe the state of the system at the conclusion of the use case execution.]
Normal Flow:	[Provide a detailed description of the user actions and system responses that will take place during execution of the use case under normal, expected conditions.
Alternative Flows:	[Document legitimate branches from the main flow to handle special conditions (also known as extensions). For each alternative flow reference the branching step number of the normal flow and the condition which must be true for this extension to be executed]
Business Rules	Use cases and business rules are intertwined. Some business rules constrain which roles can perform all or parts of a use case. Perhaps only users who have certain privilege levels can perform specific alternative flows. That is, the rule might impose preconditions that the system must test before letting the user proceed. Business rules can influence specific steps in the normal flow by defining valid input values or dictating how computations are to be performed]
Assumptions:	[List any assumptions].

Use Case Analysis

This section breaks down the main ways users will interact with the system. Each "use case" is a specific goal a user wants to achieve.

- **Use Case #1: User Registration (UC-1)**
  This use case allows a new user to create an account by providing their basic credentials so they can securely access the system as shown in Figure 14 & Table 35.

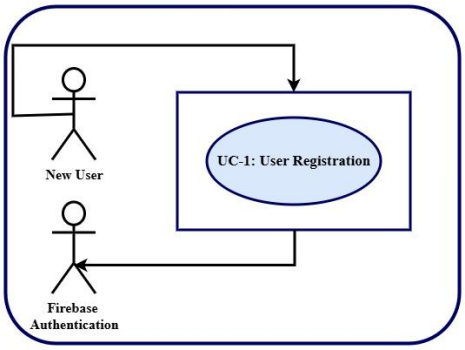


Figure 14: User Registration

Following is the table for UC-1:

Table 35: Use Case Description for "User Registration"

UC ID	UC-1
Use Case Name	User Registration
Actors	New User, Firebase Authentication
Description	Allows a new user to create an account securely.
Trigger	User selects “Sign Up” in the app.
Preconditions	App installed; user not logged in.
Postconditions	Account created in Firebase; redirected to login.
Normal Flow	1. User opens app → 2. Selects Sign Up → 3. Enters credentials → 4. Firebase validates → 5. Account created → 6. Redirect to login.
Alternative Flows	Weak password → app suggests stronger one.
Business Rules	Password must meet security criteria; email unique.
Assumptions	Stable internet; Firebase configured.

- **Use Case #2: User Login (UC-2)**
  This use case allows a registered user to sign into the system by entering valid login credentials to access their connected devices and app features as shown in **Figure *15*** & **Table *36***.

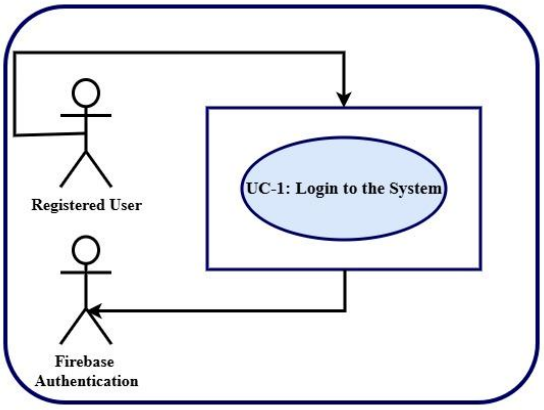


Figure 15: User Login

Following is the table for UC-2:

Table 36: Use Case Description for "User Login"

UC ID	UC-2
Use Case Name	User Login
Actors	Registered User, Firebase Authentication
Description	Allows a registered user to sign in with valid credentials.
Trigger	User selects “Login” in the app.
Preconditions	Account exists in Firebase.
Postconditions	User authenticated; dashboard displayed.
Normal Flow	1. User opens app → 2. Selects Login → 3. Enters credentials → 4. Firebase validates → 5. Dashboard displayed.
Alternative Flows	Invalid credentials → error message.
Business Rules	Only authenticated users can access devices.
Assumptions	Firebase Authentication service available.

- **Use Case #3: Manual Mode Control (UC-3)**
  This use case allows a registered user to access manual mode of the devices connected with the esp32 as shown in Figure 16 and Table 37.

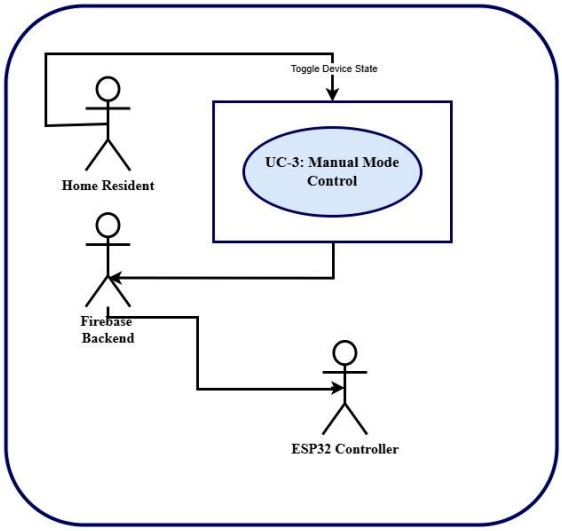


Figure 16: Manual Mode Control

Following is the table for UC-3:

Table 37: Use Case Description for "Manual Mode Control"

UC ID	UC-3
Use Case Name	Manual Mode Control
Actors	Home Resident, Firebase Backend, ESP32 Controller
Description	User directly toggles appliances ON/OFF.
Trigger	User taps toggle button in app.
Preconditions	Device paired and reachable.
Postconditions	Device state updated in Firebase; relay switched.
Normal Flow	1. User taps toggle → 2. App updates Firebase → 3. ESP32 reads update → 4. Relay switches → 5. Device state updated → 6. App refreshes UI.
Alternative Flows	Device unreachable → error message.
Business Rules	Only authenticated users can control devices.
Assumptions	Stable internet; relay functional.

- **Use Case #4: Auto Mode Activation (UC-4)**
  This use case allows a registered user to use auto mode for the devices connected to esp32 as shown in Figure 17 & Table 38.

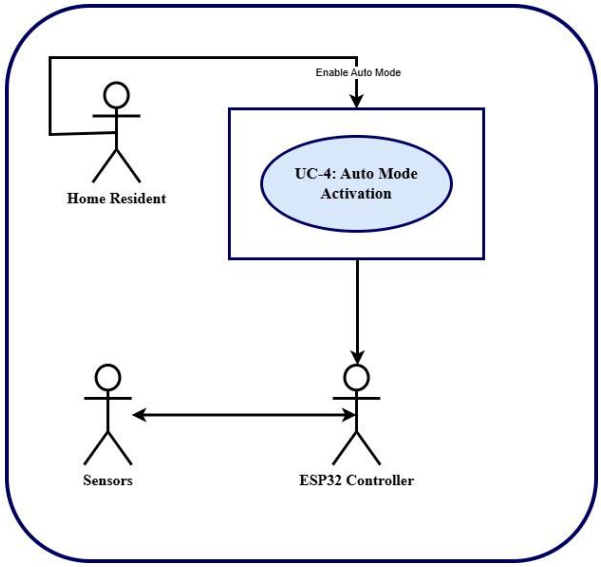


Figure 17: Auto Mode Activation

Following is the table for UC-4:

Table 38: Use Case Description for "Auto Mode"

UC ID	UC-4
Use Case Name	Auto Mode Activation
Actors	Home Resident, ESP32 Controller, Sensors (DHT11, PIR, LDR, Water Level)
Description	ESP32 automates devices based on sensor input.
Trigger	User enables Auto Mode in app.
Preconditions	Sensors connected; ESP32 online.
Postconditions	Devices controlled automatically by sensor readings.
Normal Flow	1. User enables Auto Mode → 2. Firebase updates → 3. ESP32 activates automation → 4. Sensors feed data → 5. Devices switch ON/OFF accordingly.
Alternative Flows	Sensor malfunction → system logs error.
Business Rules	Auto Mode overrides manual control.
Assumptions	Sensors calibrated; ESP32 stable.

- **Use Case #5: Schedule Mode Automation (UC-5)**
  This use case allows a new user to create an account by providing their basic credentials so they can securely access the system as shown in **Figure *18*** & **Table *39***.

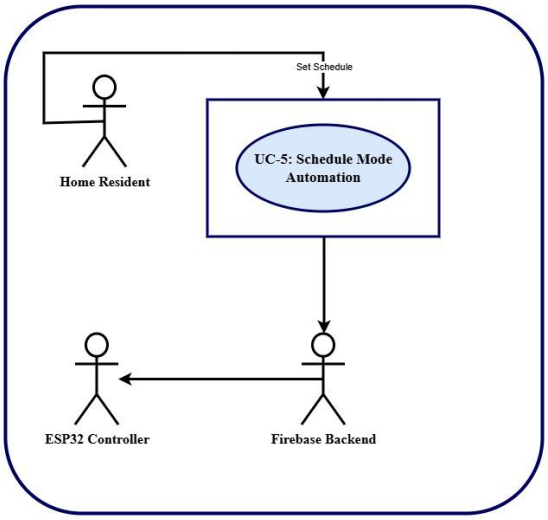


Figure 18: Schedule Mode Automation

Following is the table for UC-5:

Table 39: Use Case Description for "Schedule Mode"

UC ID	UC-5
Use Case Name	Schedule Mode Automation
Actors	New User, Firebase Authentication
Description	User defines ON/OFF times for appliances.
Trigger	User sets schedule in app.
Preconditions	User logged in; ESP32 connected.
Postconditions	Device state changes automatically at scheduled times.
Normal Flow	1. User opens app → 2. Selects device → 3. Defines ON/OFF time → 4. App saves schedule → 5. ESP32 reads schedule → 6. Relay switches at time → 7. Firebase syncs → 8. App refreshes UI.
Alternative Flows	Invalid time → app prompts correction.
Business Rules	Only authenticated users can set schedules; schedules must not overlap.
Assumptions	Accurate system clock; Firebase sync reliable.

- **Use Case #6: Monitor Usage (UC-6)**
  This use case allows a registered user to monitor the usage of each device by using the device usage screen as shown in Figure 19 & Table 40.

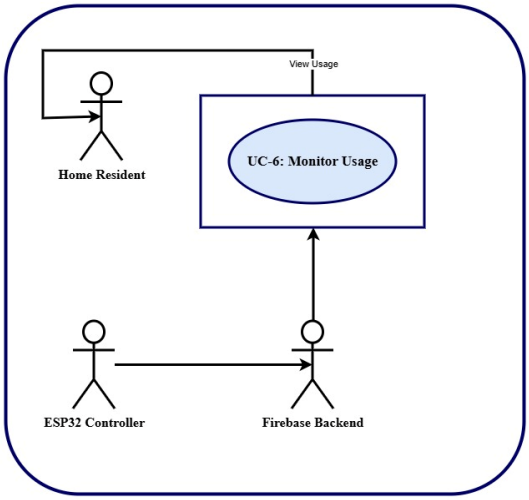


Figure 19: Monitor Usage

Following is the table for UC-6:

Table 40: Use Case Description for "Usage Monitoring"

UC ID	UC-6
Use Case Name	Monitor Usage
Actors	Home Resident, ESP32 Controller, Firebase Backend
Description	Tracks electricity consumption per device and displays usage.
Trigger	ESP32 logs usage data.
Preconditions	Devices connected; Firebase online.
Postconditions	Usage data updated in Firebase; app displays consumption.
Normal Flow	1. ESP32 measures usage → 2. Updates Firebase → 3. App retrieves data → 4. App displays usage.
Alternative Flows	Sensor error → usage not recorded.
Business Rules	Monthly reset at start of each month; default limit = 200 units.
Assumptions	Sensors calibrated; Firebase stable.

- **Use Case #7: Monthly Billing Reset (UC-7)**
  This use case allows the system to reset the monthly limit for the total allowed units to 200 and consumed units to 0 as shown in Figure 20 & Table 41.

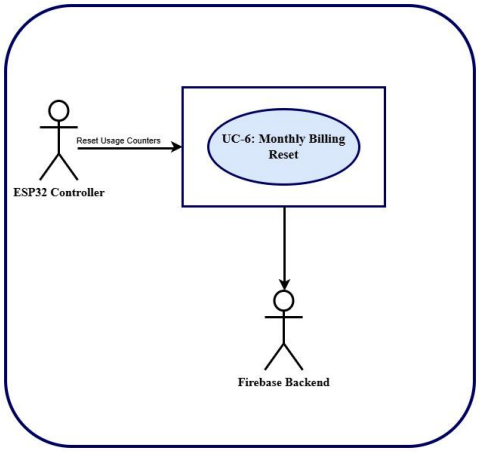


Figure 20: Monthly Billing Reset

Following is the table for UC-7:

Table 41: Use Case Description for "Monthly Billing Reset"

UC ID	UC-7
Use Case Name	Monthly Billing Reset
Actors	ESP32 Controller, Firebase Backend
Description	Resets usage data at the start of each month.
Trigger	System clock reaches new month.
Preconditions	ESP32 connected; Firebase online.
Postconditions	Usage counters reset; new month starts at 0 units.
Normal Flow	1. System clock reaches new month → 2. ESP32 resets counters → 3. Firebase updates → 4. App displays reset usage.
Alternative Flows	Firebase unreachable → reset cached locally.
Business Rules	Reset must occur automatically without user input.
Assumptions	Accurate system clock; Firebase stable.

- **Use Case #8: Trigger Safety Alert (Fire + Smoke) (UC-8)**
  This use case is concerned with the trigger of safety alarm when fire or smoke is detected through fire or smoke sensors as shown in Figure 21 & Table 42.

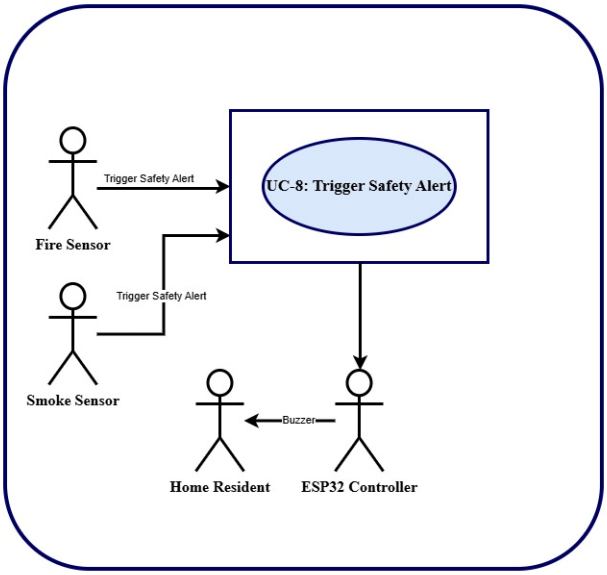


Figure 21: Trigger Safety Alert

Following is the table for UC-8:

Table 42: Use Case Description for "Trigger Safety Alert"

UC ID	UC-8
Use Case Name	Trigger Safety Alert (Fire + Smoke)
Actors	Safety Monitor, ESP32 Controller, Fire Sensor (LM393), Smoke Sensor (MQ135), Home Resident
Description	Detects fire or smoke hazards and triggers buzzer + app notification.
Trigger	Fire sensor or smoke sensor detects abnormal reading.
Preconditions	Sensors connected; ESP32 running.
Postconditions	Buzzer activated.
Normal Flow	1. Fire/Smoke sensor detects hazard → 2. ESP32 triggers buzzer → 3. Firebase updated → 4. App displays alert.
Alternative Flows	False alarm ignored; buzzer failure → app still shows alert.
Business Rules	Alerts override automation; only authenticated users can acknowledge.

Assumptions	Sensors calibrated; buzzer functional.

- **Use Case #9: Peak Hour Alert (UC-9)**
  This use case is concerned with the peak hours alert. If the motor is on during peak hours, then a buzzer will start working and there will be peak hours alert as shown in Figure 22 & Table 43.

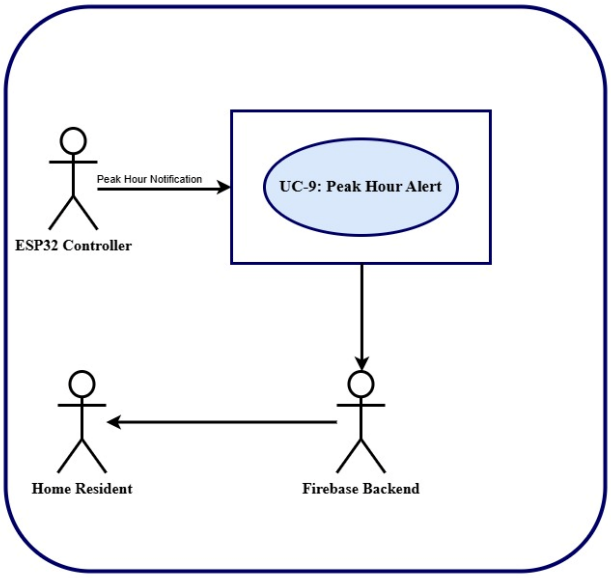


Figure 22: Peak Hour Alert

Following is the table for UC-9:

Table 43: Use Case Description for "Peak Hour Alert"

UC ID	UC-9
Use Case Name	Peak Hour Alert
Actors	ESP32 Controller, Firebase Backend, Home Resident
Description	Notifies users when high-consumption devices run during peak hours.
Trigger	System clock reaches peak time.
Preconditions	ESP32 connected; peak hours configured.
Postconditions	Buzzer activated; app notification displayed.
Normal Flow	1. System clock reaches peak → 2. ESP32 activates buzzer → 3. Firebase updates → 4. App displays notification.
Alternative Flows	Buzzer not responding → app still shows alert.
Business Rules	Peak hours predefined; alerts mandatory.

Assumptions	Accurate system clock; Firebase stable.

- **Use Case #10: Configure Peak Hours (UC-10)**
  This use case allows the registered user to configure the peak hours according to his needs as shown in Figure 23 & Table 44.

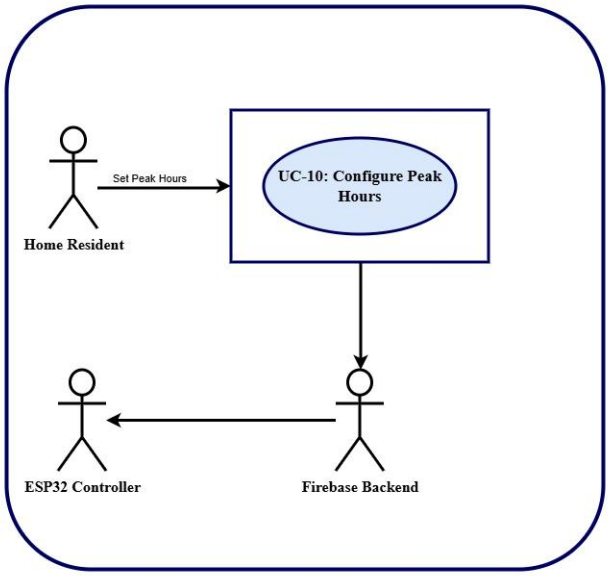


Figure 23: Configure Peak Hours

Following is the table for UC-10:

Table 44: Use Case Description for "Peak Hours Configuration"

UC ID	UC-10
Use Case Name	Configure Peak Hours
Actors	Configure Peak Hours
Description	Allows the user to define custom peak hour intervals during which alerts and notifications will be triggered if devices are running.
Trigger	User opens settings and sets peak hours in the app.
Preconditions	User logged in; ESP32 connected; Firebase online.
Postconditions	New peak hour intervals saved in Firebase; ESP32 applies updated configuration.
Normal Flow	1. User opens app → 2. Navigates to settings → 3. Selects “Peak Hour Configuration” → 4. Defines start and end times → 5. App saves settings in Firebase → 6. ESP32 reads updated configuration → 7. Alerts triggered during defined peak hours.

Alternative Flows	Invalid time range entered → app prompts correction. Firebase unreachable → settings cached locally until sync.
Business Rules	Only authenticated users can configure peak hours; peak hour ranges must not overlap; alerts mandatory during configured intervals.
Assumptions	Accurate system clock; Firebase stable; ESP32 online.

- **Use Case #11: Device Log Storage (UC-11)**
  This use case is concerned with the logging of each event in the database as shown in Figure 24 & Table 45.

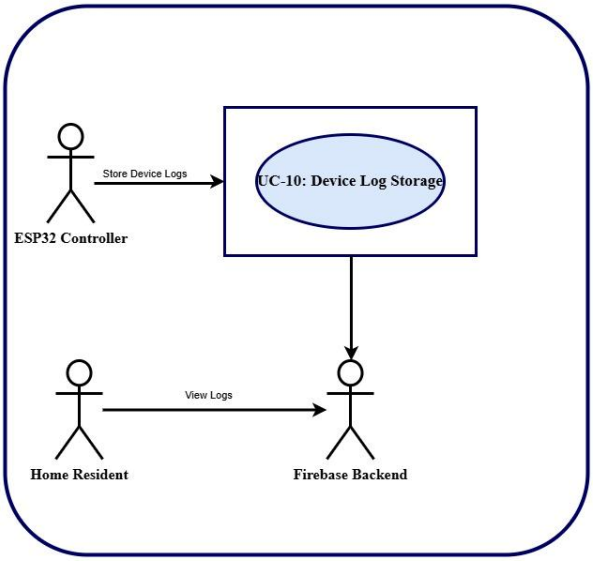


Figure 24: Device Log Storage

Following is the table for UC-11:

Table 45: Use Case Description for "Device Log Storage"

UC ID	UC-11
Use Case Name	Device Log Storage
Actors	ESP32 Controller, Firebase Backend, Home Resident
Description	Records logs of every device action (manual toggle, auto trigger, schedule execution, safety alert) and stores them in Firebase for later review.
Trigger	Any device action occurs (manual, auto, schedule, or alert).
Preconditions	Device paired; ESP32 connected; Firebase online.
Postconditions	Log entry created in Firebase; available for user review.

Normal Flow	1. Device action occurs → 2. ESP32 records event → 3. Log entry created with timestamp, device ID, action type → 4. Firebase stores log → 5. User views logs in app.
Alternative Flows	Firebase unreachable → logs cached locally until sync.
Business Rules	Logs must include timestamp, device ID, action type; logs cannot be altered by user.
Assumptions	Firebase stable; ESP32 clock accurate.

Appendix-B

Coding Standards

Appendix-B General Coding Standards & Guidelines

1. Follow a consistent variable naming convention throughout the code. E.g.
    - Snakecase (words are delimited by "_" like: variable_one)
    - Pascalcase (words are delimited by capital letters like: VariableOne)
    - Camelcase (words are delimited by capital letters except the initial word like: variableOne)
    - Hungarian Notation (describes the variable type or purpose at the start of the variable name like: arrDistributeGroup)
2. Use naming that visually describes scope like privateField, Const etc
3. Use read only/immutable when a field's value should not be changed after initialization
4. Use only get, for properties that should not be updated from outside
5. Name functions according to their functionalities.
6. Insert appropriate comments to make the code understandable to any reader. Additionally follow a consistent style to do so. E.g.

```
/* the below function will be used for the addition of two
variables*/ int Add(){
//logic of the function
}
```

   Avoid commenting on obvious things
7. Make use of indentation for indicating the start and end of the control structures along with a clear specification of where the code is between them.
8. Follow consistent naming convention for files and folders.
9. Follow proper structure for classes
10. Group code entities logically into projects/packages/modules/folders
    a. Separate logical layers of application into different modules/services/utilities etc.
    b. User separate files for each class, struct, interface, enum etc. Name of the file and the enclosing entity must be same. E.g., class Employee in Employee.cs/Employee.java
11. Define and use everything within the minimum scope possible
12. Use proper access modifier for all code entities if required
13. Code entities should have maximum cohesion and least coupling possible.
14. Follow DRY law.
    a. Do not repeat code.
    b. A piece of knowledge should exist only in one place within the codebase/application
    c. Reuse code as much as possible
    d. Always write short methods
    e. Single method should not have too many logic, long conditional flow or too many parameters
15. Strictly follow Single Responsibility Principle (SRP) when writing methods, classes, modules, projects, packages, or any other code entities.
16. Write classes and other code entities that are easy to extend without modification.
17. Handle exceptions
18. Log exception and other significant event details
19. Follow a consistent convention for logging all over the application

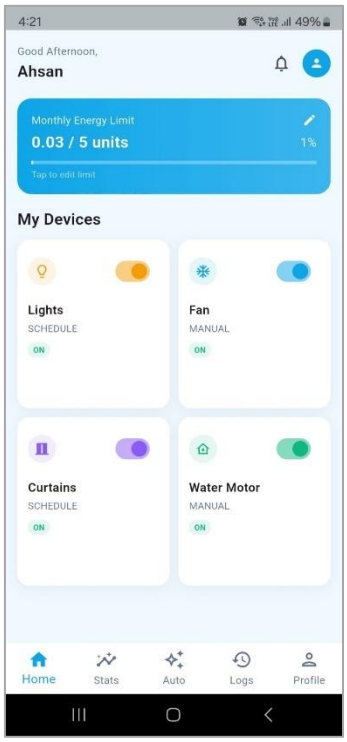
Appendix C

Protoype

Appendix-C Application Prototype

C.1 Splash Screen

C.2 Dashboard



C.3 Usage Monitoring